

REȚELE LOCALE



IONESCU Valeriu Manuel

Universitatea din Pitesti
2022

CURS 1

Cunoștințe fundamentale de rețele de calculatoare necesare pentru cursul de rețele locale de calculatoare

Comunicațiile prin internet se realizează prin multiple tipuri de comutație:

- comutația de circuite (circuite fizice sau virtuale);
- comutația de pachete (în Rețele de calculatoare se folosește multiplicarea statistică pentru a folosi eficient canalul de comunicații) – sloturile alocate în funcție de cerere – rezultă folosire eficientă;
- comutația de celule (în rețele ATM) – similară cu comutația de pachet, însă dimensiunea pachetului transmis este fixă.

În marea majoritate a cazurilor, rețelele locale utilizează comutația de pachete. Deoarece multiple protocoale de comunicație contribuie la realizarea schimbului de date între Modele folosite în RC (arhitecturi):

- OSI (7 straturi) – este un model teoretic care permite înțelegerea transformării datelor la traversarea rețelelor, însă nu este folosit în practică;
- TCP/IP (4 straturi) – este folosit în practică. Prezintă niveluri similare cu OSI, însă echivalența dintre acestea nu este completă.

Încapsularea datelor: segment, pachet, frame, biți, date.

Echipamente folosite în RC și repartizarea acestora pe nivelurile OSI:

- la nivelul fizic: hub-uri, repetitoare, transceiver, convertitoare;
- la nivel date: switch/bridge;
- la nivel rețea: router.

Topologii fizice și logice în RC:

- fizice – așezarea/dispunerea fizică a calculatoarelor în rețea (stea, magistrală, inel, mesh, etc.);
- logice – se referă la modul în care pachetele sunt transmise: topologia de tip broadcast și cea de tip token passing.

Medii de transmisie folosite în rețele de calculatoare: cablu coaxial, cablu torsadat, fibră optică, wireless.

Structura plăcii de rețea (hard și soft) și funcționarea ei

Modul de funcționare al algoritmului CSMA/CD folosit în rețelele de calculatoare bazate pe Ethernet.

Adresele IP:

- Clasificare
- Mască
- Gateway

- Tabela de routere
- Calcul AR și AB

Protocoloale de routere:

- RIP – routing information protocol (routere pas cu pas)
- OSPF

TCP vs. UDP

TCP – orientat pe conexiune;

UDP – nu are conexiune.

Servicii în rețele de calculatoare:

- WEB
- DNS
- DHCP
- FTP
- TelNet
- SSH

Metode pentru securizarea Rețelelor de calculatoare:

- autentificarea;
- criptarea datelor: Tipuri de criptare: cu cheie publică ; cu cheie privată.
- algoritmi: RSA - pentru criptare cu cheie publică;
AES – pentru criptare cu cheie privată.

Traficul în rețele de calculatoare

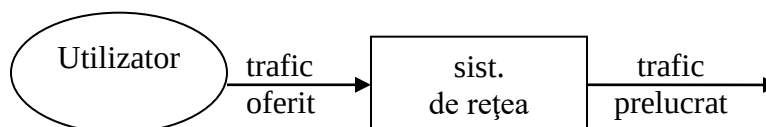
Traficul este o componentă a rețelilor ce nu apare într-o prezentare structurală. El este o componentea dinamica care se face simțită prin ocuparea circuitelor și dispozitivelor.

Traficul nu poate fi stocat → prelucrarea se face în timp real.

Traficul poate fi:

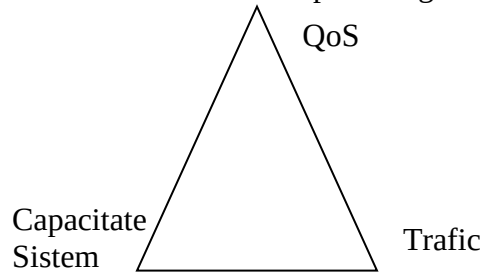
- oferit – se produce prin solicitările utilizatorilor;
- prelucrat – circulă printr-o rețea de calculatoare, dimensionat corespunzător.

Traficul diferă în funcție de modul de comutare folosit.



Un sistem trebuie să răspundă la momentul proiectării la una din următoarele întrebări:

1. Fiind dat un sistem de capacitate cunoscuta, cu un anumit nivel de trafic oferit, care este calitatea serviciului care poate fi garantat?



2. Cum trebuie dimensionata capacitatea sistemului dacă știu traficul oferit și calitatea dorită?

3. Pentru o capacitate a sistemului și o calitate dorită, care este traficul maxim suportat?

Există trei factori ce trebuie luați în calcul în proiectarea unei rețele: QoS, trafic și capacitatea sistemului.

Sistemul poate fi un singur element, întreaga rețea sau o parte a acesteia.

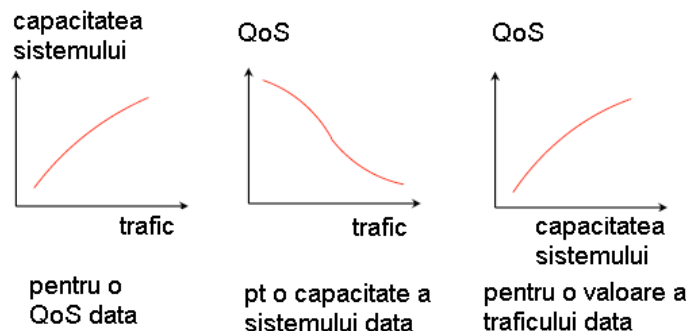
Traficul caracterizat prin fluxuri, conexiuni, biți, pachete (depinde modul în care se realizează schimbul de date) – depinde de perioada de timp (transferul).

Calitatea serviciului în rețele de calculatoare se referă la posibilitatea asigurării de priorități diferite pentru aplicații, utilizatori, fluxuri de date sau garantarea unui anumit nivel de performanță. Calitatea serviciului poate fi descrisă atât din punct de vedere al utilizatorului (în acest caz privește latența, variația latentei, viteza de transmisie, întreruperea conexiunii) cat și din punctul de vedere al sistemului când se folosește termenul de performanță în utilizarea rețelei.

Există trei elemente interdependente după cum urmează:

- QoS
- trafic
- capacitate sistem

Relația între cei 3 factori:



Pentru a descrie relația din punct de vedere cantitativ sunt necesare metode matematice.

Modele de trafic

Modelele de trafic sunt de tip probabilistic (stochastic) deoarece momentul efectuării schimbului de date nu poate fi prevăzut.

Elementul probabilistic este determinat de variabile aleatoare precum numărul de conexiuni, numărul de pachete din buffer și dimensiunea pachetelor.

S-au elaborat modele probabilistice care încearcă să descrie parțial sistemele reale. Scopul practic al elaborării modelelor este proiectarea corectă a rețelelor de calculatoare. Aceasta are două aspecte: planificarea rețelelor și managementul rețelelor.

Planificarea cuprinde dimensionarea, optimizarea și analiza performanțelor.

Managementul se referă la operarea eficientă, remediarea erorilor și poate și contabilitatea atunci când este vorba de un trafic plătit.

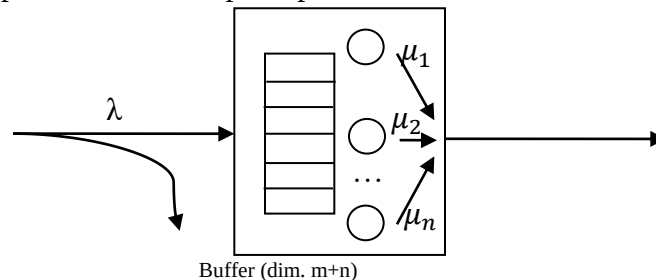
Există 3 tipuri de sisteme model:

- **cu pierderi**
- **cu cozi de așteptare**
- **cu partajare**

Acestea sunt modele simple care în realitate se pot combina pentru a crea modele complexe

Considerăm următoarele elemente care descriu un model de trafic:

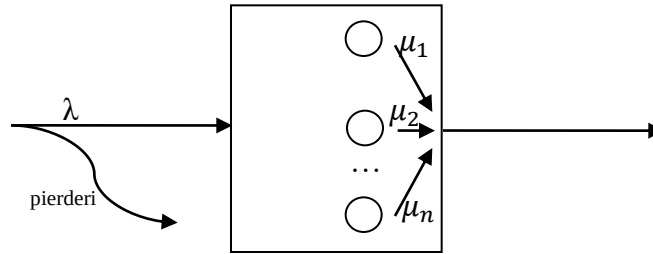
- λ - rata de sosirea a utilizatorilor
- $1/\lambda$ - perioada între sosirile utilizatorilor
- μ - rata serviciului
- $1/\mu$ - timpul mediu de așteptare pentru un utilizator.



Bufferul poate avea n poziții de lucru și m poziții de așteptare.

1. Sistem cu pierderi

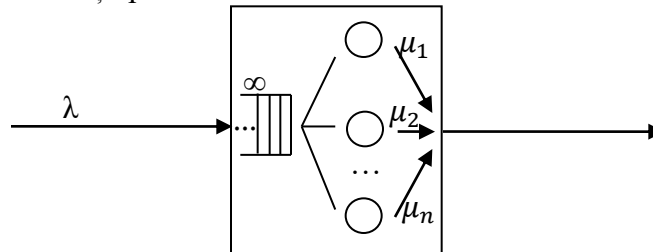
- Nu există poziții de așteptare ($m=0$);
- Utilizatorii peste capacitate sunt pierduți, dacă sistemul este plin.
- de interes este probabilitatea ca sistemul să fie plin când sosesc utilizatorii



2. Sistemul infinit

- Acesta este un model teoretic care are un număr infinit de servere care procesează cererile clienților;
- Nu există pierderi și nici așteptare;
- Este folosit pentru a determina limita superioară a unui sistem real.

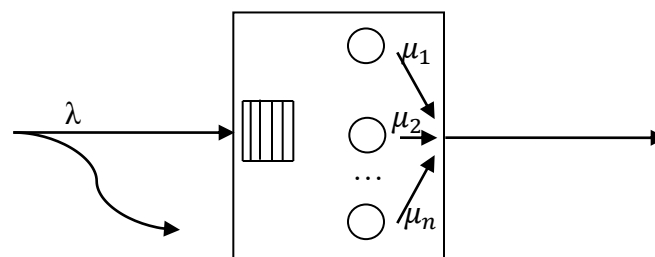
3. Sistem pur cu așteptare



- Nu are pierderi;
- Are un număr finit de poziții de lucru;
- Dimensiunea bufferului de așteptare este infinită;
- Dacă toate serverele sunt ocupate când sosește un client, atunci acesta ocupă o poziție de așteptare.

Dezavantaj: - unii clienți ar putea să aștepte foarte mult.

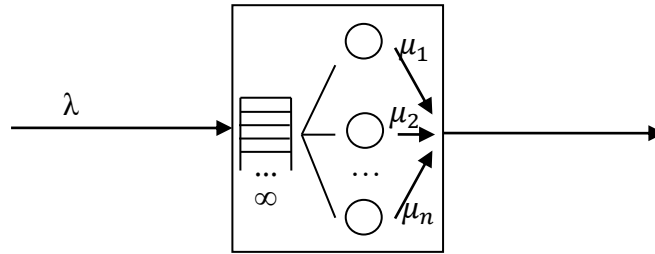
4. Model complex: sistem de așteptare cu pierderi.



m – finit

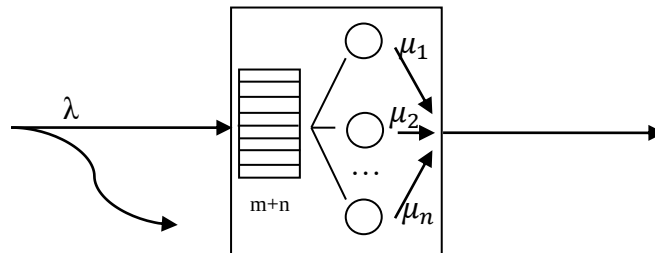
- Dimensiunea bufferului (m) este finită;
- Dacă bufferul este gol, clienții sunt puși în așteptare;
- Dacă bufferul este plin, clienții sunt pierduți.

5. Sistem pur cu partajare.



- Niciun client nu este pierdut;
 - Niciun client nu trebuie sa astepte;
 - Daca numarul utilizatorilor este mai mic sau egal decat numarul de servere, fiecare are propriul server;
 - Daca numarul de clienti este mai mare decat numărul de servere (n) atunci rata de serviciu este impartita → serverele proceseaza pe rand si clientii suplimentari.
- O problema poate fi intarzierea procesarii cererilor.

6. Sistemul cu partajare si pierderi.



- Dimensiunea bufferului este finita;
- Niciun client nu asteapta, dar unii pot fi pierduti.

Legea lui Little

Considerând un sistem stabil cu rata medie a sosirilor λ , $\bar{N}$ numarul mediu de clienti din sistem si $\bar{T}$ timpul petrecut in medie de un client in sistem, următoarea relație este valabila: $\bar{N} = \lambda * \bar{T}$.

Aceasta lege poate fi reformulata astfel: Timpul mediu petrecut intr-un sistem este egal cu timpul mediu de așteptare plus cu timpul mediu necesar pentru primirea serviciului.

Aceasta relatie este importanta deoarece atata faptul ca factori precum: nu au nici o de influenta in relatia dintre aceste elemente. Legea este utilizata in gestionarea sistemelor complexe (formate din mai multe sub-sisteme) pentru stabilirea timpului de traversare sau al ritmului de productie. In retelele de calculatoare, de exemplu, poate fi vorba despre timpul necesar unui client sa primeasca serviciul de la server (de la momentul in care cererea clientului ajunge la server pana la momentul in care datele procesate pleaca de la server catre client).

Traficul in rețele de calculatoare

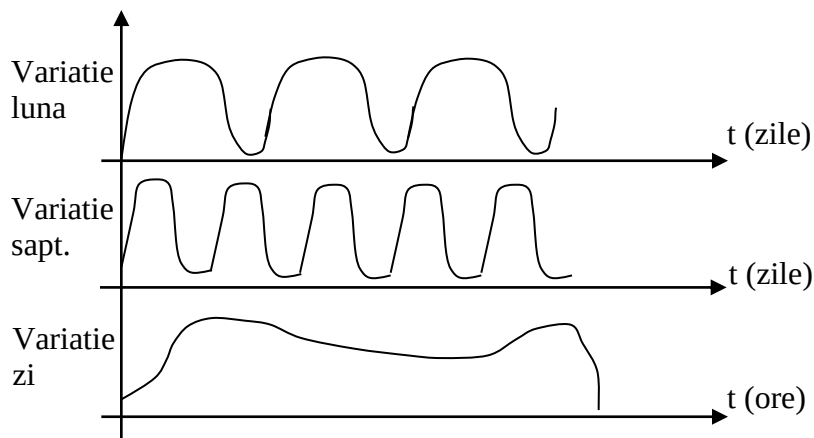
Traficul este cantitatea informatională în mișcare care implică noțiunea de debit. Aspectele traficului sunt: intensitate, fluiditate și situații de blocare sau saturare.

Traficul are diferite forme:

- trafic oferit (dpdv al providerului),
- trafic prelucrat,
- trafic refuzat,
- trafic de așteptare,
- trafic de comandă.

Clasificarea traficului

Traficul în rețea nu este constant, el având variații predictibile la nivel de an, săptămână, zi.



Pentru dimensionarea traficului este necesară o estimare a acestuia; în estimare se vor considera în special variațiile predictibile și mai puțin variațiile nepredictibile.

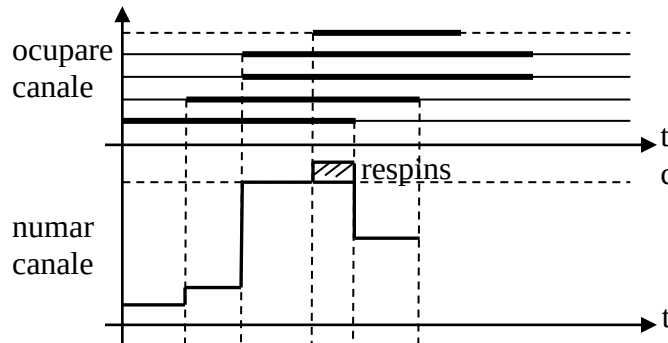
Variații nepredictibile: variații la nivel de minut, variații aleatoare pe termen lung privind de exemplu modificarea profilului utilizatorului, variații datorate evenimentelor neprevăzute.

Trafic:

- Comutație de circuite;
- Comutație de pachete:
 - La nivel de pachet (IP);
 - Flux:
 - UDP;
 - TCP.

Traficul în comutație de circuite – **este un sistem pur cu pierderi în care un server corespunde unui canal**, rata de serviciu depinde de timpul mediu de ocupare,

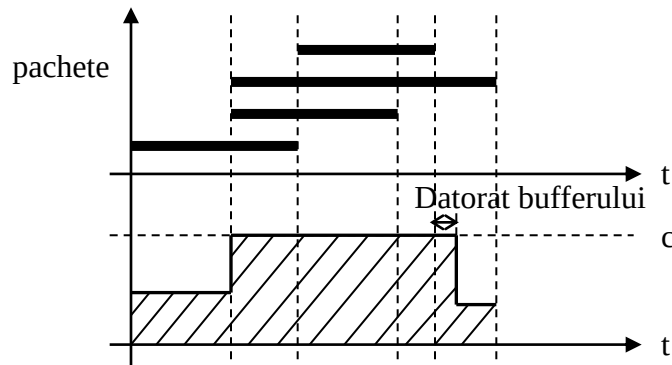
numarul de servere depinde de capacitatea liniei, cand toate canalele sunt ocupate apelurile entare sunt respinse.



In comutia de circuite se foloseste tehnica TDM (time division multiplexing).

Traficul la nivelul pachetelor

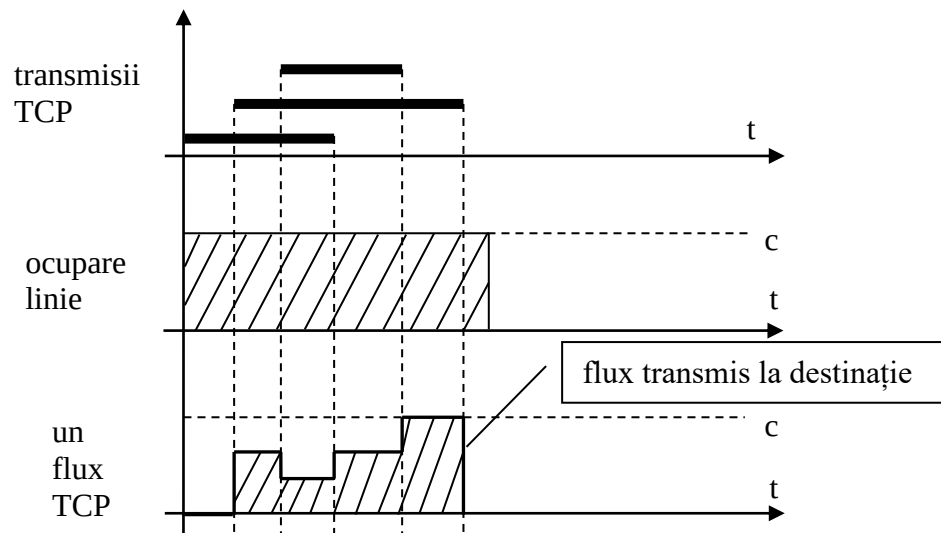
- **Este un sistem cu asteptare bazat pe un singur server;**
- Rata de serviciu depinde atat de capacitatea liniei cat si de lungimea medie a pachetelor;
- In comutatia de pachete utilizatorii concura pentru resursele retelei;
- Cand linia este ocupata pachetele noi sunt trecute in buffer; daca acesta este plin, pachetele sunt pierdute.



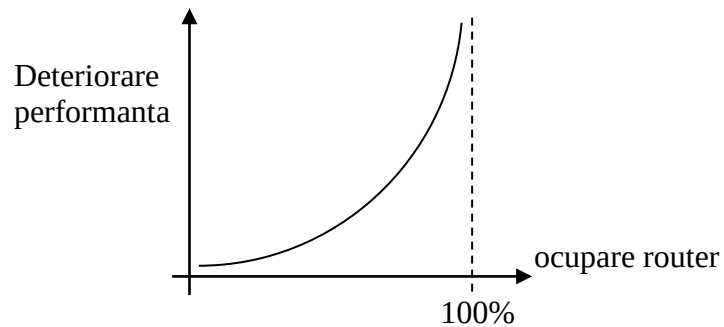
Traficul la nivel de flux.

Traficul TCP se mai numeste si **trafic elastic** deoarece isi poate redimensiona fluxul in timpul realizarii schimbului de date. Modelul este un sistem cu partajare deoarece nu exista controlul intrarilor deci niciun flux de intrare nu este respins.

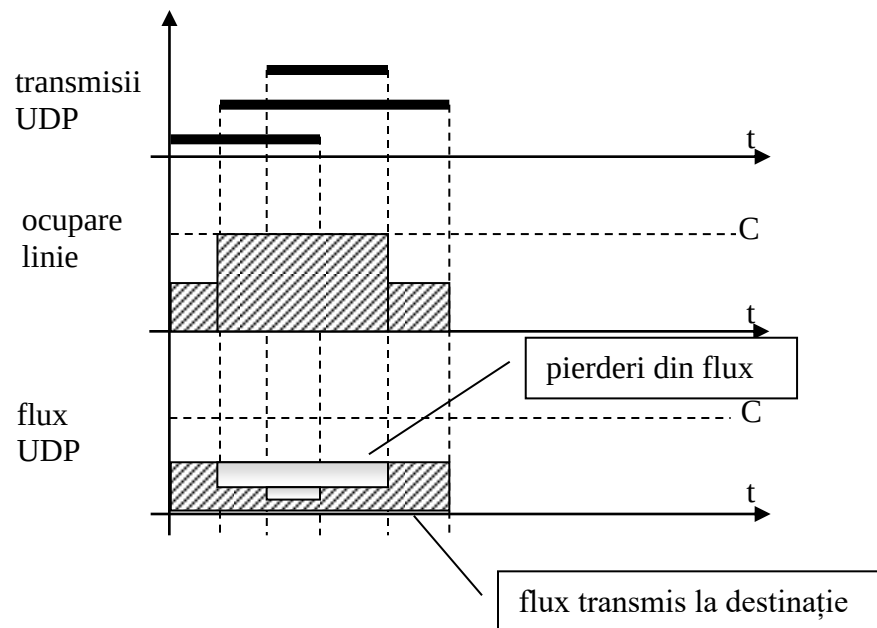
Rata de serviciu depinde de capacitatea liniei si mărimea fluxului.



Se observa cum in cazul a trei transmisii TCP simultane banda disponibila se împarte in mod egal la cele trei fluxuri, fără sa aibă loc pierderi de date. Tehnica folosita de fluxul TCP pentru creștere este creșterea aditiva (treptat) si descreșterea multiplicativă (cu 1/2).



- Traficul de flux tip UDP (se mai numeste si **streaming**):
- **Modelul este cel al unui sistem infinit;**
 - Rata de prelucrare depinde de lungimea fluxului;
 - Nu exist bufferi → traficul suplimentar este pierdut uniform din toate streamurilor;
 - Nu exista control al intrărilor.



Spre deosebire de fluxurile TCP care pot detecta congestia liniei de comunicație (și în consecință își vor redimensiona), fluxurile UDP care însumate depășesc capacitatea liniei de comunicație nu se vor modifica, consecința fiind că se vor pierde date din toate aceste fluxuri.

În figura se observă cum fiecare flux UDP are pierderi, însă, deoarece în UDP nu este implementat un mecanism de detecție a apariției congestiei rețelei, acestea vor persista cât timp este depășită capacitatea canalului de transmisie. Considerând că un flux UDP ocupă 50% din capacitatea liniei de comunicație, la apariția mai multor fluxuri UDP, doar o mică parte din date vor mai ajunge la destinație, celelalte fiind pierdute.

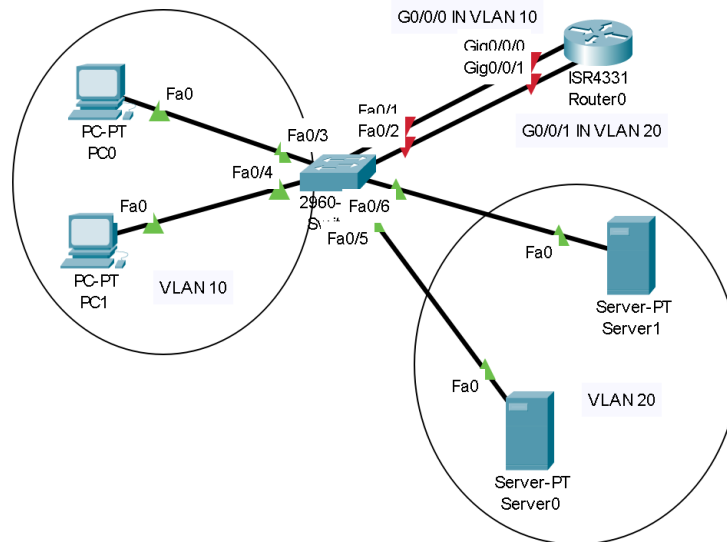
CURS 3 (PART TWO)

VLANs

Switch-ul este un echipament de rețea care funcționează la nivelul 2 al modelului OSI care concentrează traficul de rețea provenit de la echipamentele utilizatorilor. Acest echipament are multe port-uri (de la 8 până la 48 de porturi) care îi permit să conecteze un număr mare de utilizatori. Un Switch identifică dispozitivele conectate prin intermediul adreselor fizice MAC și folosește protocolul Ethernet pentru a permite oricărui două echipamente conectate la un switch să poată comunica.

VLAN-urile (Virtual Local Area Network) ne permit să separăm din punct de vedere logic porturile unui switch astfel încât traficul dintr-un VLAN să nu poată trece în alt VLAN. O utilitate este, de exemplu, separarea traficului intern al unei firme (contabilitate, dezvoltatori software) de traficul extern al acesteia (de exemplu cel oferit clienților sau partenerilor firmei), fără a avea nevoie de mai multe dispozitive switch.

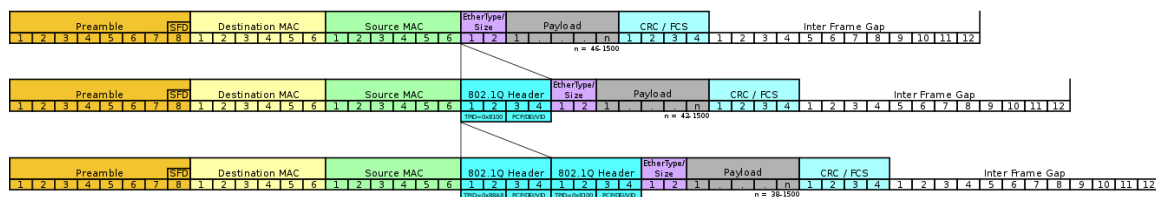
Izolarea traficului dintr-un VLAN inseamna ca pentru fiecare VLAN va fi o retea distincta, deci un singur domeniu de broadcast. Pentru a conecta doua VLAN-uri este nevoie de un router. In figura urmatoare, un router conecteaza VLAN 10, care contine PC-urile PC1 si PC0 dar si interfata Fa0/1 a switch-ului, de VLAN 20, care contine PC-urile Server1 si Server0 dar si interfata Fa0/2. Reteaua VLAN-lui 10 include prin urmare G0/0/0, PC1 si PC0 iar VLAN-lui 20 include G0/0/1, Server1 si Server0.



Pentru identificare, VLAN-urile folosesc in VLAN ID care poate sa ia valorile:

- Standard VLAN ID – 1 – 1005
- Extended VLAN ID – 1006 – 4094

Protocolul care specifica operarea VLAN-urilor este 802.11q care difera de Ethernet (IEEE 802.3) prin adaugarea in antetul frame-ului a unui camp ce identifica VLAN-ul (12 biti) si alte elemente specifice.



Suplimentar exista standardul 801.ad care este folosit in special de providerii de servicii de internet pentru a transmite prin retelele interne (care au VLAN-uri) traficul clientilor care la randul este de tip 802.1q (cu identificatorul de VLAN). In acest mod clientii pot conecta VLAN-uri aflate in locatii diferite conectate de reseaua providerului de servicii de internet.

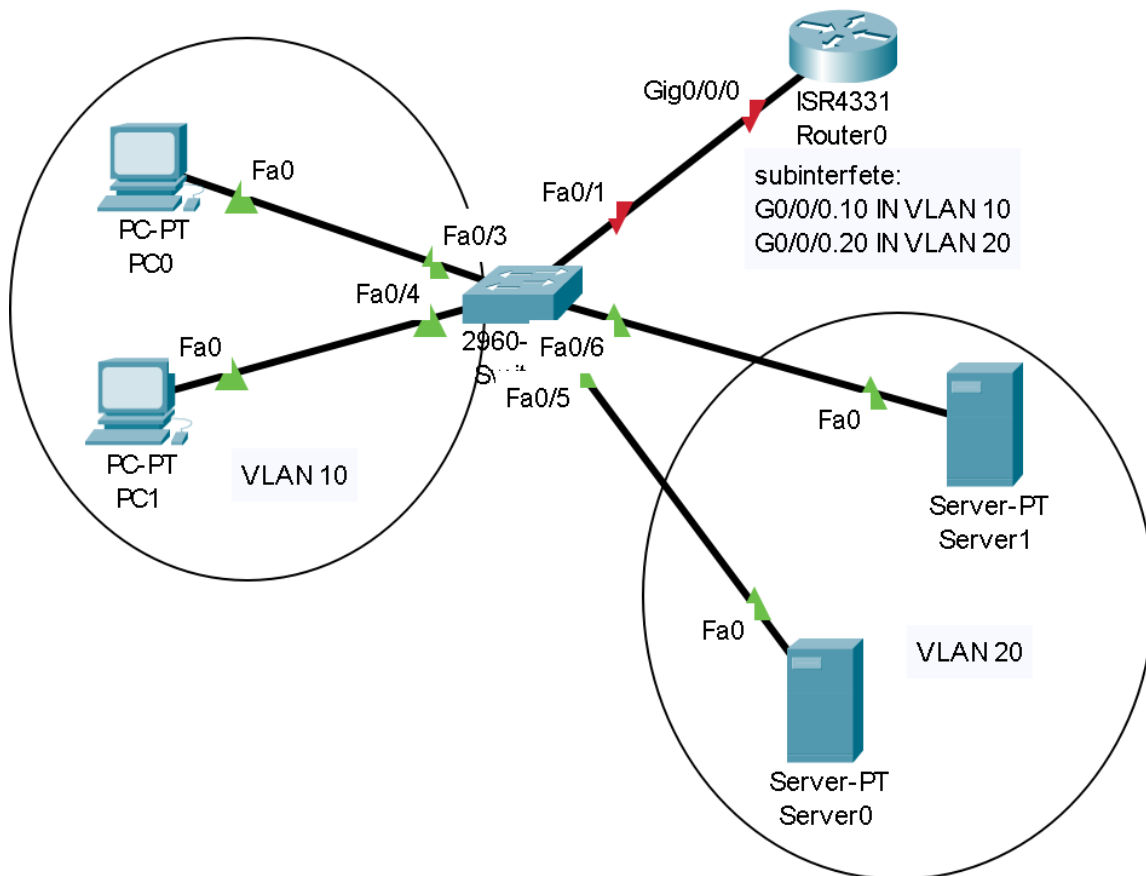
Avantaje utilizare VLAN:

- Crestere performanta prin reducerea domeniului de broadcast (mai putine echipamente sunt intr-o retea)

- securitate sporita deoarece traficul este izolat in interiorul VLAN-ului. Orice incercare de a trimite trafic dintr-un VLAN in alt VLAN in interiorul switch-ului nu este posibila.
- flexibilitate in configurarea retelelor deoarece plasarea unui calculator intr-un VLAN sau altul (deci intr-o retea sau alta) se poate face prin software, fara a necesita interventie fizica asupra conexiunilor existente
- reducerea costurilor prin folosirea unui numar redus de switch-uri cat si reducerea numaului de fire necesare pentru conectarea echipamentelor.

Exista insa o problema: pe rutere numarul porturilor este realtiv redus, fiind frecvent prezente echipamente care integrat au un singur port Ethernet. Prin urmare nu putem avea cate o interfata de router pentru fiecare VLAN.

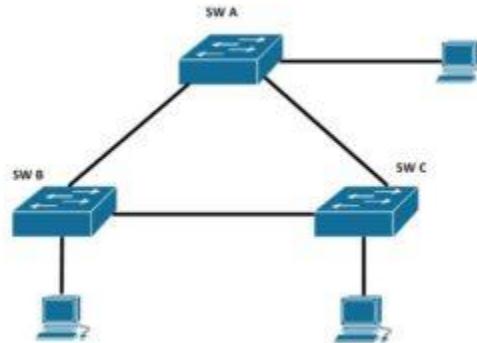
Prin urmare a fost introdus pe langa modul ACCES de functioarea al portului obisnut al unui switch (utilizat in conectarea echipamentelor utilizator) si un mod numit TRUNK. Acesta se configureaza un portul de pe switch dinspre router si permite traversarea sa de catre traficul din mai multe VLAN-uri. Fiecare trafic este marcat corespunzator conform protocolului 802.1q. Pe ruter se va configura o subinterfata pentru fiecare VLAN care traverseaza linia de trunk. Acest lucru se vede in figura urmatoare.



Un caz special este cel al introducerii de trafic Ethernet pe o linie prin care circula trafic de tipul 802.1q. In acest caz traficul Ethernet este introdus in retea

Spanning Tree Protocol

În rețelele locale (LAN) poate apărea un fenomen interesant (dacă avem o topologie similară cu cea de mai jos). Când se generează/trimită trafic (mai exact, un mesaj de tip broadcast (i.e. ARP)), comportamentul default al Switch-urilor este de a trimite pachetul pe toate interfețele mai puțin pe cea de pe care a venit acest pachet. Asta înseamnă că fiecare Switch va trimite traficul către PC-urile direct conectate și către un Switch vecin.



Acest comportament duce la formarea unei **bucle de rețea**. O buclă de rețea (**loop**) are un impact negativ asupra rețelei (și a echipamentelor):

- *Incarcarea lățimii de bandă (BW)*
- *Incarcarea procesorului fiecărui echipament de rețea*
- *Posibil downtime în rețea (cel mai rău caz)*

Spanning-Tree Protocol (STP) este un mecanism de protecție împotriva buclilor de rețea. Acestea apar în rețelele în care există mai multe căi posibile către o anumită destinație. **STP** ne garantează (din orice punct al rețelei) **o singură cale** (cea mai rapidă) **pană la destinație**.

Impactul pe care îl pot avea buclele asupra rețelei este foarte mare. CPU-ul poate ajunge la 100%, existând posibilitatea ca Switch-urile să se restarteze. STP este soluția pentru aceste probleme.

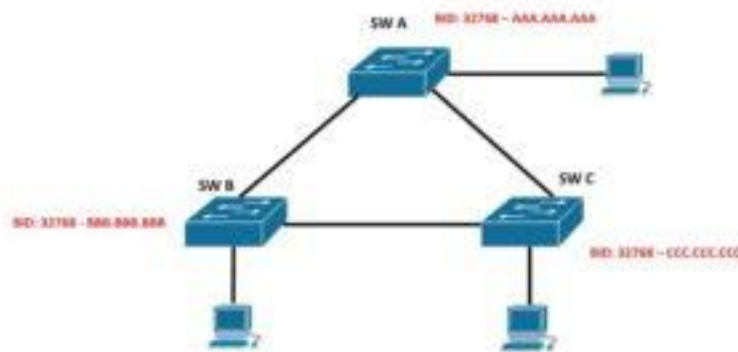
Pentru a crea un *cale fără buclă în rețea*, în STP trebuie ales un punct central al rețelei. Acest punct central poartă denumirea de **Root Bridge**. Acesta se alege, în mod dinamic, în rețea pe baza celui mai mic **BID** (Bridge ID) care este compus din:

- *Prioritate (1 – 65535)*
- *Adresa MAC a Switch-ului*

BID = Prioritate + MAC

Astfel fiecare Switch din rețea va avea un BID (Bridge ID) unic.

! Switch-ul cu **cel mai mic BID** va fi ales **Root Bridge**.



Sa luam exemplul de mai jos:

BID SW 1:

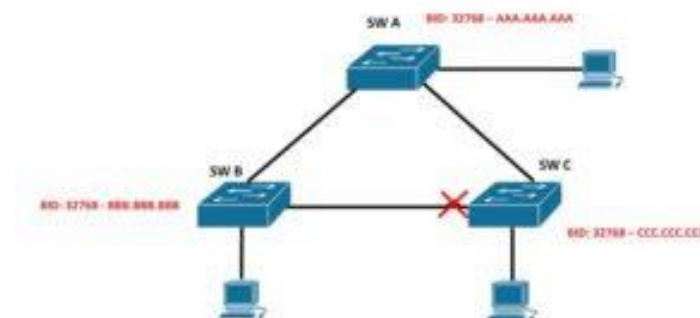
- 32768 (prioritatea default) + AAA.AAA.AAA (adresa MAC a Switch-ului A)

BID SW 2:

- 32768 (prioritatea default) + BBB.BBB.BBB (adresa MAC a Switch-ului B)

BID SW 3:

- 32768 (prioritatea default) + CCC.CCC.CCC (adresa MAC a Switch-ului C)

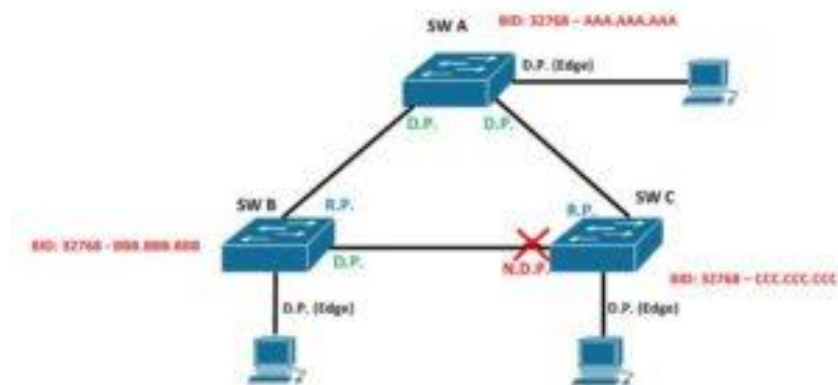


Avand acest BID-uri (fictive) STP-ul isi va face treaba si va alege un punct central al retelei (aka. Root Bridge). Odata ce se alege Root Bridge-ul, celelalte Switch-uri vor cauta **cea mai scurta** cale catre acesta si **vor bloca link-urile redundante**.

Tipuri de Porturi in STP

Odata ce este stabilit Root Bridge-ul, Switch-urile trebuie sa stabileasca fiecare port de ce tip va fi. In **STP** exista mai multe tipuri de porturi, precum:

- **Designated Port (DP)** – starea “clasica” a unui port
- **Root Port (RP)** – portul duce catre Root Bridge
- **Non-Designated Port (NDP)** – portul este blocat



Root Bridge-ul va avea intotdeauna **toate porturile Designated (DP)** si vor trimite activ trafic. Celelalte Switch-uri din LAN vor avea cate **un singur port de tipul Root Port (RP)** care vor indica calea catre Root Bridge. Ca regula de baza in STP, fiecare link trebuie sa aiba 1 singur DP.

Daca se pune problema ca 2 porturi sa fie DP => unul dintre ele trebuie sa fie in starea NDP (adica blocat): ex. link-ul dintre SWB si SWC.

SWB si SWC trebuie sa decida pe link-ul dintre ele, *CINE VA BLOCA PORTUL*. In acest caz, cel care are **BID-ul mai MIC (SWB)** va avea portul DP, iar SWC va avea **portul blocat (NDP)**.

Starea Porturilor in STP

Un **port poate trece prin mai multe stari** inainte de a fi functional. Acestea sunt:

- *Disabled* – portul este oprit (#shutdown)
- *Blocked* – nu trimite trafic pe port
- *Listening* – asculta traficul care vine pe port
- *Learning* – incepe sa invete adrese MAC pe port
- *Forwarding* – trimite trafic (starea “clasica”)

By default, un port (inca de la pornire) trebuie sa treaca prin fiecare din aceste stari (incepand cu Blocking pana la Forwarding).

- Un port de tipul **DP** (Designated Port) se afla in starea **Forwarding** (el trimite activ trafic).
- Un port de tipul **RP** (Root Port) se afla in starea **Forwarding** (el trimite activ trafic).
- Un port de tipul **NDP** (Non-Designated Port) se afla in starea **Blocking** (nu trimite trafic).

CURS 4

Calitatea serviciului (Qos- quality of service) ~Definirea calității serviciului si a performantei rețelei~

Calitatea serviciului in rețele de calculatoare se refera la posibilitatea asigurării de priorități diferite pentru aplicații, utilizatori, fluxuri de date sau garantarea unui anumit nivel de performanta.

Efectul performantelor serviciului d.p.d.v. al utilizatorului = calitatea serviciului.

Performanta rețelei reprezintă capacitatea rețelei de a realiza funcțiile de comunicație între utilizatori:

- Accesul la mediul de transmisie;
- Stabilitatea comunicației;
- Calitatea comunicației.

Calitatea serviciului are in vedere următoarele elemente:

1. Adaptarea rețelei la nivelul de performanta solicitat.
2. Accesul la comunicații de voce, video si date.

Performanta rețelei, dpdv al providerului, trebuie sa asigure controlul urmatorilor parametri:

- Latime de banda (band width $\neq$ through put = viteza de transfer instantanee);

Exp. O linie de 10 Mb/sec latime de banda

Transfer de 5 Mb/sec

Through put este mai mic decat latimea de banda (exceptie este atunci cand se foloseste compresia).

- Intarzierea;
- Variatia intarzierii (jitter);
- Pierderea de pachete.

In schimbul de date, entitatile implicate sunt urmatoarele:

1. Clientul – este partea care plateste pentru serviciile asigurare.
2. Furnizorul de retea – asigura conectivitatea, dar nu neaparat serviciile.
3. Furnizorul de servicii – asigura restul serviciilor (exp. DNS – serviciul de nume).
4. Utilizatorul – cel care foloseste serviciile oferite.

~Cerinte de performanta impuse rețelelor IP~

Latimea de banda nu este un element dinamic, deci nu suporta criterii de performanta.

Intarzierea este diferenta între timpul la care este receptionat si timpul la care a fost transmis. Ea este determinata de:

- Modul de serializare a informatiei (dimensiunea pachetului);
- Timpul de propagare (de ordinul μ s-elor);
- Timpul de comutatie care creste in functie de marimea cozii de asteptare (router).

Jitterul (variatiia intarzierii) – pentru reducerea acestuia se folosesc bufferi. Din pacate bufferii, maresc intarzierea.

Pierderea de informatie este determinata de erori de bit sau de pierderi de pachete si poate fi imbunatatita prin marirea compresiei datelor.

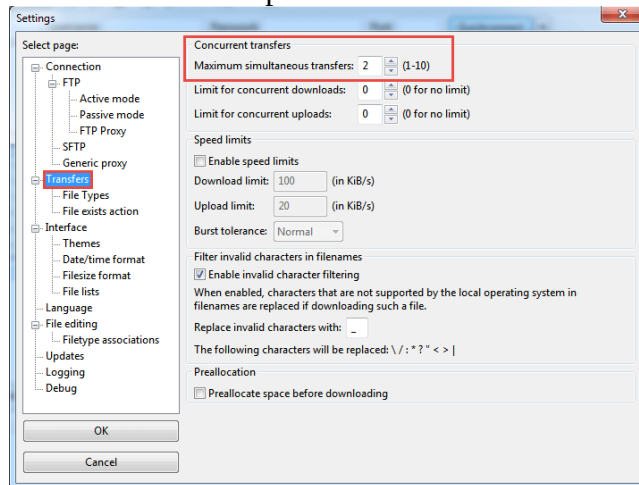
In concluzie, putem clasifica cele doua elemente componente ale calitatii serviciului astfel:

1. QoS orientat pe utilizator, vizeaza serviciile retelei; se manifesta intre punctele de acces ale serviciului.
2. Network performance este orientat pe provider (furnizor); priveste elementele de conectivitate si se manifesta intre elementele de conectivitate.

Tehnici de asigurare a QoS solicitata de aplicatie:

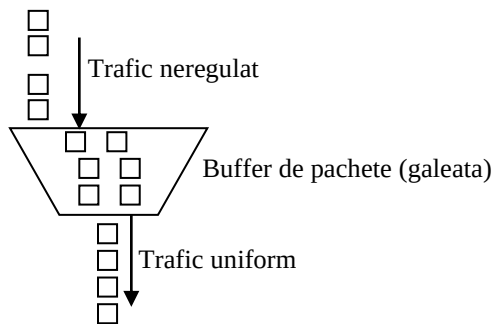
1. Supra-aprovizionarea este rareori aleasa deoarece implica costuri suplimentare, sau nu este disponibila.
2. Memorarea temporara – este o solutie pentru jitter in detrimentul latentei. Nu sunt afectate fiabilitatea si latimea de banda.
3. Trafic shaping (modelarea traficului) – este metoda preferata de provideri pentru a monitoriza / controla fluxul de date. Foloseste tehnici multiple pentru evitarea congestiei / aglomerarii retelei.

Unele servere au implementate aceste mecanisme (exemplu Filezilla server):



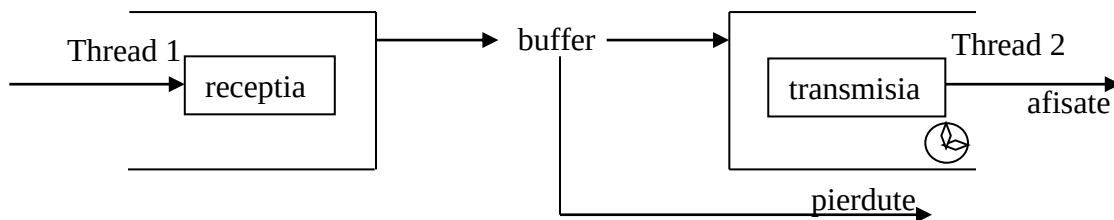
4.

Metode de control al traficului intr-un nod al retelei
Algoritmul Leaky-bucket (galeata sparta).

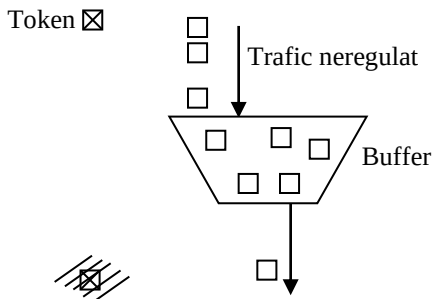


Aceasta solutie presupune folosirea unui buffer pentru a uniformiza traficul. Pachetele ies din buffer cu o anumita frecventa independenta de rata de intrare. Daca bufferul se umple, pachetele sunt pierdute.

Aplicatia server are doua componente:



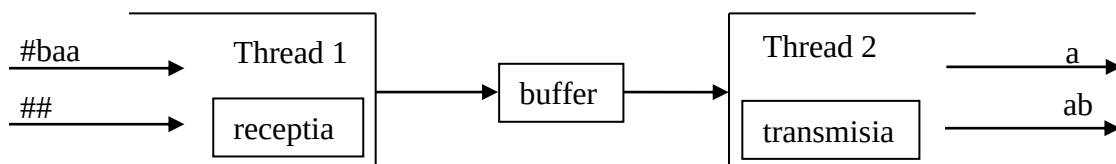
Algoritmul Token-bucket



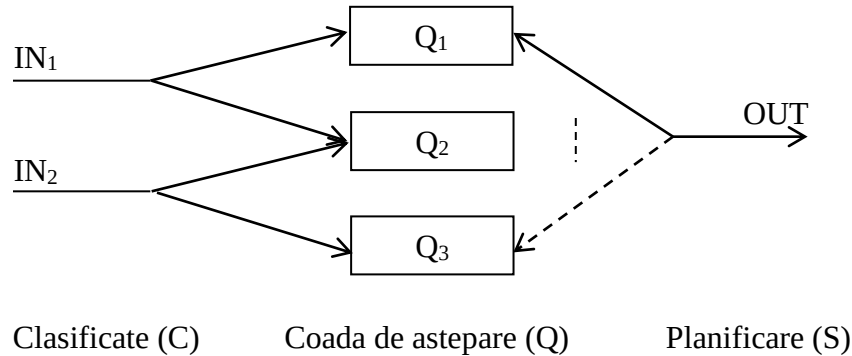
Bufferul de pachete scoate date doar daca se primesc caractere de control (tokeni / jeton) pentru un pachet ieseit se distruge un token.

Algoritmul ignora/elimina tokenurile daca bufferul este plin.

Implementarea permite controlul iesirii de la sursa; din buffer nu se pierd pachete; este algoritmul folosit la comunicatia intre routere.



5. Rezervarea resurselor implica stabilirea resurselor necesare la momentul conexiunii si utilizarea lor exclusiva.
 - Se foloseste un protocol dedicat RSVP.
6. Controlul accesului – routerule primesc cereri de QoS specificate prin parametrii QoS doriti si alti parametri aditionali precum dimensiunea pachetelor.
7. CQS (classification Queing Scheduling) – aceasta metoda presupune clasificarea pachetelor, introducerea in cozile de asteptare (modul in care se alege din fiecare coada de asteptare sa fie transmis) si programarea iesirii acestora.



CURS 5

Mecanisme de implementare a QoS

DiffServ este un model pentru furnizarea de QoS în Internet prin diferențierea traficului. Metoda de cel mai bun efort folosită pe internet încearcă să ofere cel mai bun serviciu posibil, în funcție de fluxul de trafic diferit, mai degrabă decât să încerce să diferențieze fluxul și să ofere un nivel mai ridicat de serviciu pentru o parte din trafic. DiffServ încearcă să ofere un nivel îmbunătățit de servicii în mediul de efort existent prin diferențierea fluxului de trafic. De exemplu, DiffServ va reduce latența în traficul care conține voce sau streaming video, oferind în același timp cel mai bun serviciu pentru traficul care conține transferuri de fișiere. Pachetele sunt marcate de dispozitivele DiffServ la granițele rețelei cu informații despre nivelul de serviciu cerut de acestea. Alte noduri din rețea citesc aceste informații și răspund în consecință pentru a oferi nivelul de serviciu solicitat.

IntServ este un alt model pentru furnizarea de QoS în rețele. IntServ se bazează pe construirea unui circuit virtual pe internet folosind tehnica de rezervare a lățimii de bandă. Solicitățile de rezervare a lățimii de bandă provin de la aplicațiile care necesită un anumit nivel de serviciu. Conform acestui model, fiecare router din rețea trebuie să implementeze IntServ și fiecare aplicație care necesită garanție de serviciu trebuie să facă o rezervare. Când lățimea de bandă este rezervată pentru o anumită aplicație, aceasta nu poate fi reatribuită pentru o altă aplicație. Routerule dintre expeditor și destinatar determină dacă pot suporta rezervarea făcută de aplicație. Dacă nu o pot suporta, ei anunță destinatarul. Altfel, trebuie să direcționeze traficul către receptor. Prin urmare, în această metodă, routerule își amintesc proprietățile fluxului de trafic și, de asemenea, îl

supraveghează. Sarcina de a rezerva căi ar fi foarte costisitoare într-o rețea aglomerată, cum ar fi Internetul.

IntServ – mecanism ce are algoritmi orientati pe flux si foloseste protocolul RSVP (resource reservation protocol).

Dezavantajele modelului IntServ sunt:

- Fiecare flux trebuie semnalizat in retea, lucru problematic in cazul unui numar mare de fluxuri;
- Echipamentele necesita bufferi de dimensiuni mari;
- Schimbul de informatie intre routere este complex.
- Necesita acorduri intre provideri

Desi IntServ are avantaje privind eficienta rezervarii resurselor, costurile suplimentare au facut ca in prezent sa fie raspandite metodele DiffServ.

DiffServ clasifica fluxurile in functie de calitatea serviciului oferit.

DiffServ:

- Marcheaza pachetelor prin scrierea campurilor ToS (type of service) – ne necesita rezervarea de resurse;
- Traficul este extrem de redus.

0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding

In prezent, sunt definite mai multe niveluri pentru calitatea serviciului (sunt numerotate de la 0 la 5 si apar in pachetul IP):

0 – servicii de timp real; retea asigura cozi de asteptare cu servire preferentiala; 0 avem jitter sub 100 ms.

- 1 – tot pentru servicii de voce si timp real, de prioritate redusa; sub 400 ms.
 - 2 – serviciile de semnalizare in retea (confirmari, acknowledgement etc.), sub 100 ms (cu pierderi de pachete).
 - 3 – servicii de date cu interactivitate redusa; sub 400 ms. (cu pierderi de pachete).
- Obs. 2 si 3 – retea asigura cozi de asteptare separate prioritare la pierderi;

- 4 – aplicații cu pierderi mici (transferuri scurte, videostreaming); cozile de aplicații sunt lungi și direcționarea pachetelor se face pe orice traseu disponibil; jitter sub o secundă
- 5 – servicii (restul serviciilor IP). nicio restricție (există pierderi de pachete)

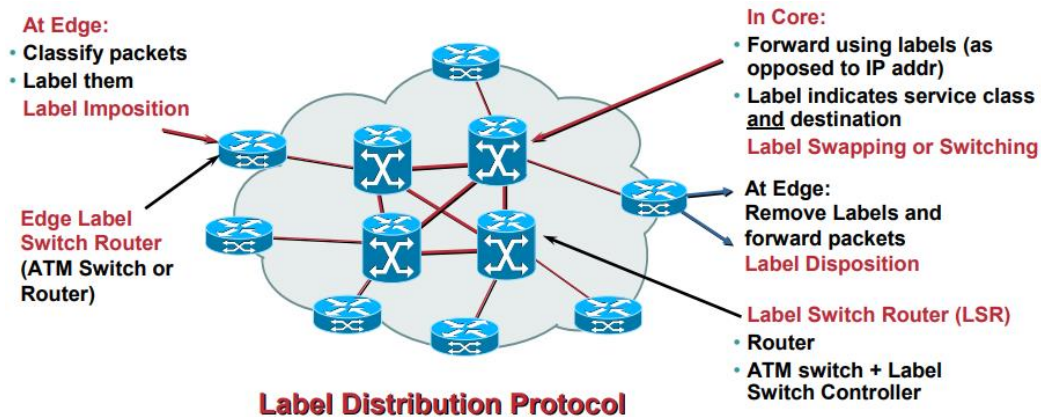
Importanța calității serviciului pentru diferite servicii IP:

Posta electronică (email)	Mică	Mică	Mică	Mare
Traficul web	Medie	Mare	Mică	Mare
Remote desktop	Medie	Mare	Medie	Mare
Videostreaming	Mare	Mare	Mare	Medie
Audio conferință	Mare	Mare	Mare	Medie
	Latime de banda	Latenta	Jitter	Pierderi

Rețelele IP au fost proiectate să fie de tipul best-effort, adică pierderea unui pachet să nu fie semnalizată însă se încearcă transmiterea acestora în proporție cât mai mare. Din acest motiv, protocoalele de nivel superior precum TCP pot asigura un anumit nivel al QoS.

MPLS

În prezent, se folosește din ce în ce mai mult tehnologia MPLS pentru asigurarea calității serviciului. Inițial, MPLS a fost proiectată drept comutație de pachete orientată pe conexiune.



- Create new services via flexible classification
- Provides the ability to setup bandwidth guaranteed paths
- Enable ATM switches to act as routers

	(QoS and ECN)	o m-of-Stack	
--	---------------	--------------	--

Aceste pachete etichetate MPLS sunt comutate după o căutare/comutație (label lookup) de etichetă în loc de o căutare în tabelul IP. După cum sa menționat mai sus, atunci când a fost conceput MPLS, căutarea etichetelor și comutarea etichetelor au fost mai rapide decât o tabelă de rutare sau o căutare RIB (Routing Information Base), deoarece puteau avea loc direct în structura comutată și nu în CPU.

Totuși, prezența unei astfel de etichete trebuie să fie indicată routerului/comutatorului. În cazul cadrelor Ethernet, acest lucru se realizează prin utilizarea valorilor EtherType 0x8847 și 0x8848, pentru conexiuni unicast și, respectiv, multicast.

Label switch router

Un router MPLS care efectuează rutarea numai pe baza etichetei se **label switch router (LSR)** sau **transit router**. Acesta este un tip de router situat în mijlocul unei rețele MPLS. Este responsabil pentru comutarea etichetelor utilizate pentru rutarea pachetelor.

Când un LSR primește un pachet, folosește eticheta inclusă în antetul pachetului ca index pentru a determina următorul hop pe label-switched path (LSP) și o etichetă corespunzătoare pentru pachet dintr-un tabel de căutare. Vechea etichetă este apoi îndepărtată din antet și înlocuită cu noua etichetă înainte ca pachetul să fie direcționat înainte.

Label edge router

Un Label edge router (LER, cunoscut și sub numele de edge LSR) este un router care funcționează la marginea unei rețele MPLS și acționează ca puncte de intrare și de ieșire pentru rețea. LER-urile împing o etichetă MPLS pe un pachet de intrare și o scot de pe un pachet de ieșire. Alternativ, sub penultimul hop popping această funcție poate fi efectuată de LSR conectat direct la LER.

Când transmite o datagramă IP în domeniul MPLS, un LER utilizează informații de rutare pentru a determina eticheta adecvată care trebuie aplicată, etichetează pachetul în consecință și apoi transmite pachetul etichetat în domeniul MPLS. De asemenea, la primirea unui pachet etichetat care este destinat să iasă din domeniul MPLS, LER-ul dezlipеște eticheta și redirecționează pachetul IP rezultat folosind regulile normale de redirecționare IP.

Provider router

În contextul specific al unei rețele private virtuale (VPN) bazate pe MPLS, LER-urile care funcționează ca routere de intrare și/sau de ieșire către VPN sunt adesea numite routere PE (Provider Edge). Dispozitivele care funcționează doar ca routere de tranzit sunt numite în mod similar routere P (furnizor). Responsabilitatea unui router P este semnificativ mai ușoară decât cea a unui router PE, astfel încât pot fi mai puțin complexe și pot fi mai fiabile din această cauză.

Label Distribution Protocol

Etichetele sunt distribuite între LER și LSR folosind Label Distribution Protocol (LDP). LSR-urile dintr-o rețea MPLS schimbă în mod regulat informații de etichetă și accesibilitate între ele folosind proceduri standardizate pentru a construi o imagine completă a rețelei pe care o pot folosi apoi pentru a transmite pachete.

Routing

Când un pachet neetichetat intră în routerul de intrare și trebuie să fie transmis unui tunel MPLS, routerul determină mai întâi forwarding equivalence class (FEC) pentru pachet și apoi inserează una sau mai multe etichete în antetul MPLS nou creat al pachetului. Pachetul este apoi transmis următorului router hop pentru acest tunel.

Antetul MPLS este adăugat între antetul stratului de rețea și antetul stratului de legătură al modelului OSI.

Când un pachet etichetat este primit de un router MPLS, eticheta cea mai de sus este examinată. Pe baza conținutului etichetei, se efectuează o operațiune de swap, push (impunere) sau pop (eliminare) pe stiva de etichete a pachetului. Routerurile pot avea tabele de căutare predefinite care le spun ce fel de operație să facă pe baza etichetei de sus a pachetului primit, astfel încât să poată procesa pachetul foarte rapid.

- Într-o operațiune de schimb, eticheta este schimbată cu o nouă etichetă, iar pachetul este transmis pe calea asociată cu noua etichetă.

- Într-o operațiune de împingere, o nouă etichetă este împinsă deasupra etichetei existente, „încapsulând” efectiv pachetul într-un alt strat de MPLS. Aceasta permite rutarea ierarhică a pachetelor MPLS. În special, aceasta este utilizată de VPN-urile MPLS.

- Într-o operațiune pop, eticheta este scoasă din pachet, ceea ce poate dezvălui o etichetă interioară dedesubt. Acest proces se numește „decapsulare”. Dacă eticheta declanșată a fost ultima din stiva de etichete, pachetul „părăsește” tunelul MPLS. Acest lucru este de obicei făcut de routerul de ieșire, dar consultați Penultimate Hop Popping (PHP) de mai jos.

În timpul acestor operațiuni, conținutul pachetului de sub stiva MPLS Label nu este examinat. Într-adevăr, ruterele de tranzit trebuie de obicei doar să examineze eticheta cea mai de sus de pe stivă. Redirecționarea pachetului se face pe baza conținutului etichetelor, ceea ce permite „redirecționarea pachetelor independentă de protocol” care nu trebuie să se uite la o tabelă de rutare dependentă de protocol și evită potrivirea costisitoare a prefixului IP cel mai lung la fiecare hop.

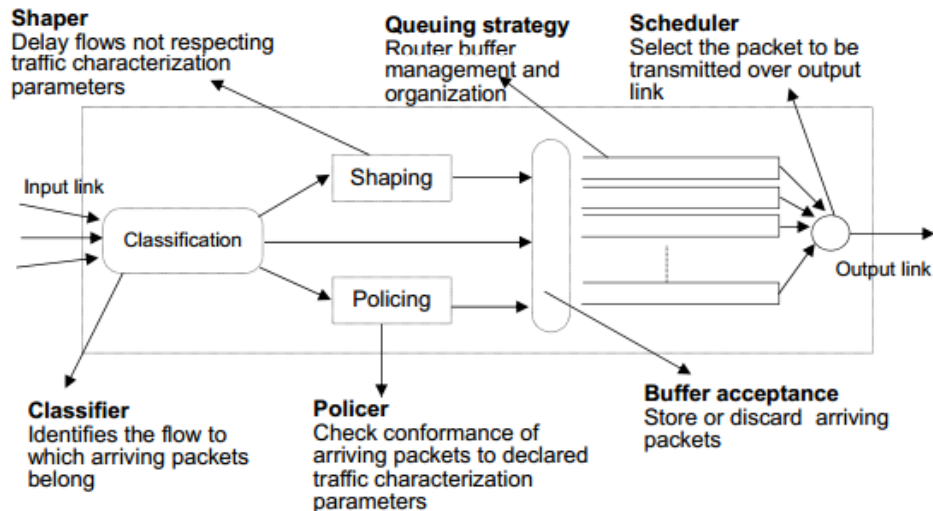
Deoarece MPLS simplifică mult direcționarea pachetelor putând să grupeze fluxuri de același tip prin etichete similare; posibilitatea direcționării atât a pachetelor IP cât și a ATM mecanismul MPLS a permis implementarea garanției calității serviciului.

~Metode de planificare a pachetelor~

Planificarea pachetelor este ultima etapă de procesare înainte de transmiterea pachetelor către interfața de ieșire.

Planificatorul este un arbitru / bloc de decizie ce hotaraste care pachet memorat in cozile de asteptare va fi trimis spre iesire.

QoS-capable router



The implementation of Quality of Service mechanisms is a critical element to solving the problems of handling the increasing volume and disparate types of traffic seen on today's public and private networks. Whenever a mission critical application – whether voice, video, or data – is being delivered over a network, Quality of Service traffic shaping and policing can assist network managers in maintaining the best network performance and good-put for applications during periods of network congestion. QoS offers the following core principles:

- Classification and Marking
- Policing
- Queuing and Dropping

Both Class of Service (CoS) and ToS IP Precedence (or DSCP) are Quality of Service methods used to manage the network during congestion.

Class of Service allows the network administrator to identify the order of priority that the incoming traffic be processed for transmission when the switch or network is congested. As an example, a weighted round robin (WRR) method prevents head-of-line blocking. Network administrator can setup the queues to match various business requirements and priority levels. The IEEE 802.1p standard recommends the Class of Service traffic categories described in Table 1.

Priority	Type of Traffic	Equivalent Application Traffic
0 (lowest)	Best effort	Ordinary LAN priority traffic
1	Background	File transfers, games, AIM
2	(spare)	Not used
3	Excellent effort	For critical users applications
4	Controlled load	For important applications
5	Video, < 100ms latency and jitter	Video application or mix with Voice
6	Voice, < 10ms latency and jitter	Voice application
7 (highest)	Network control	Critical traffic to maintain network

Table 1. Class of Service traffic parameters

IP Precedence allows the network administrator to prioritize IP traffic. Using the ToS IP Precedence or DSCP fields, the network administrator can design the edge switch or router to effectively classify, mark, and process packets according to the application requirements. Most critical application traffic takes precedence over any routine traffic processing. The IP Precedence consists of 3 bits, while the DSCP consists of 6 bits (64 possible values).

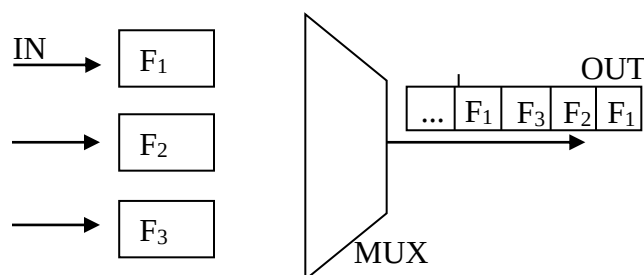
Se urmaresc in luarea deciziilor:

- *Asigurarea vitezei;*
- *Low balancing intre conexiuni concurente;*
- *Garantarea ratei de pierderi;*
- *Garantarea intarzierii si jitterului.*

Metode de planificare:

1. FIFO

- Routerelor tradiționale au o singura coada de așteptare asociată interfeței de ieșire;



- Algoritmul este simplu (costuri reduse);
- nu ofera protecție împotriva fluxurilor non-standard
- Nu poate garanta întârzierea și viteza de transfer;

In routerelor moderne ce suportă funcții de calitate a serviciului, planificatorul controlează ieșirea prin disciplina de planificare integrată în router.

2. Processor Sharing

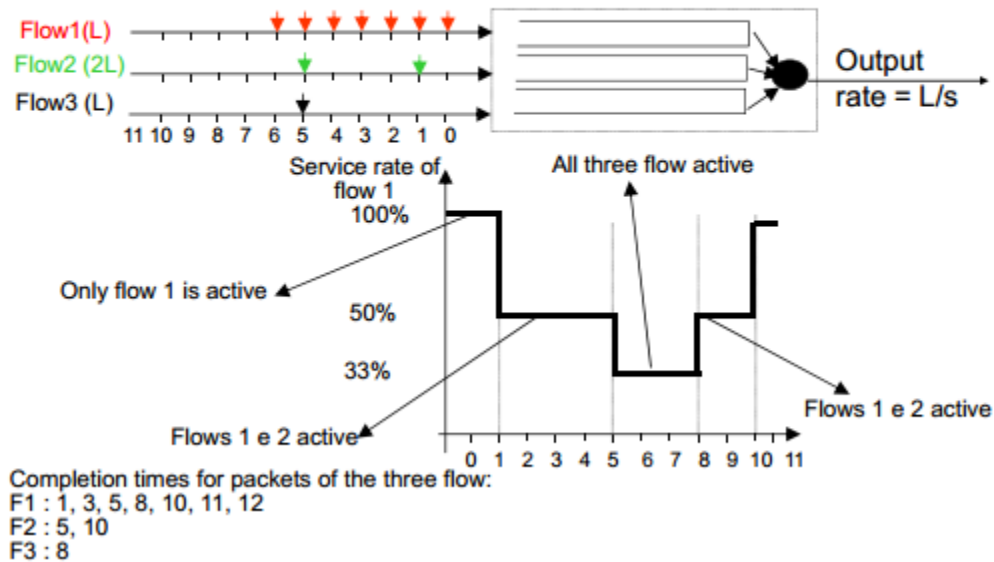
Fiecare coada este servita conform unui model fluid in care traficul este infinit divizibil, dimensiunea pachetului nu conteaza, daca n clienti sunt serviti de un canal de capacitate C, fiecare vede capacitatea C/N

Este un nmodel ideal!

Se asigura serviciu indiferent de dimensiune pachet

$\text{rata}[i] = \text{rata totala} / \text{flux}$

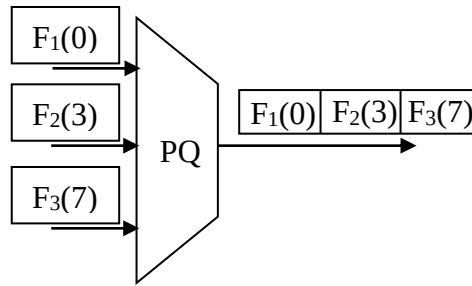
Processor Sharing: example



De exemplu 3 traficuri de tip TCP nu vor imparti exact traficul in 1/3 ci aproximativ aceasta valoare...

3. Priority Queuing (PQ)

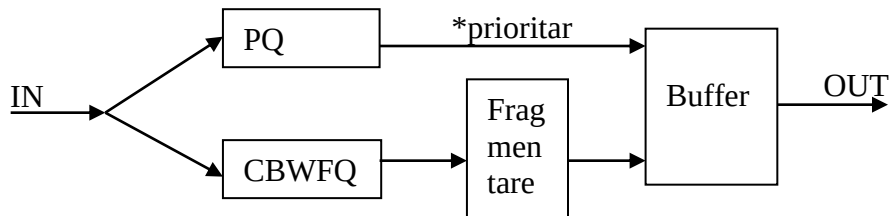
- Implica ordonarea cozilor descrescator in functie de prioritatea acestora
- Aceste routere analizeaza campul prioritate din antetul IP. In IP v6 acesta are valori de la 0-7 in care 0 este asociat traficului neclasificat, iar 7 este asociat traficului de prioritate maxima (exp. Traficul de routere);
- Se trece la servirea unei cozi cu prioritate mai mica doar daca cozile cu prioritate mai mare sunt goale.
- Dezavantajul este ca acest mod de lucru poate bloca accesul la mediul de transmisie pentru cozile cu prioritate mai mica.



- Aceasta schema este strict prioritară. Planificatorul servește toate pachetele din coada de așteptare cu prioritate cu prioritate mare înainte de a trece la coada de așteptare cu următoarea prioritate (mai mică).
Dezavantaj:
 - poate bloca fluxurile cu prioritate redusă;
 - timpul mediu de întârziere este dependent de valoarea MTU (maximum transmission unit).

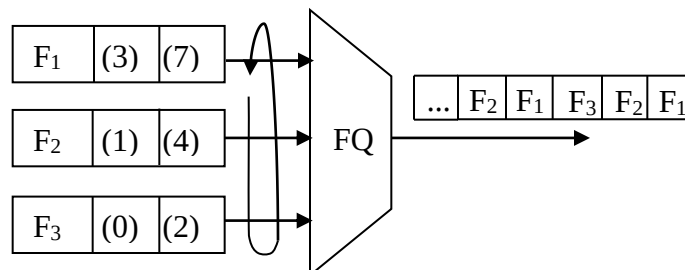
Load Balancing Queuing

Acest mecanism combină metoda de clasificare după prioritate și metode de prioritarizare în funcție de clasă. Traficul asociat cozilor tratate cu algoritmul PQ este servit prioritar înaintea celui CBWFQ.



4. FQ (Fair Queuing)

- Pentru această tehnică / metodă planificatorul își adaptează dinamic disciplina de servire a cozilor de așteptare în raport cu numărul de biți transmiși recent de fiecare coadă. Poate garanta asigurarea lățimii de bandă minimă sau maximă;
- În fiecare etapă de transmisie orice flux are dreptul să transmită un pachet. Extragerea se realizează octet cu octet.
- The advantage over conventional first in first out (FIFO) or priority queuing is that a high-data-rate flow, consisting of large packets or many data packets, cannot take more than its fair share of the link capacity.
- Coada de așteptare va fi reorganizată pentru fiecare flux în parte;



- FQ asigura un tratament egal pentru toate cozile de asteptare;
- Daca dimensiunea pachetului este fixa (retele ATM) se asigura o alocare echitabila a vitezei de transmisie;
- Daca pachetele au dimensiuni diferite sunt favorizate pachetele de dimensiuni mari.

Fair queuing attempts to emulate the fairness of bitwise round-robin sharing of link resources among competing flows. Packet-based flows, however, must be transmitted packetwise and in sequence. Fair queuing selects transmission order for the packets by modeling finish time for each packet as if they could be transmitted bitwise round robin. The packet with the earliest finish time according to this modeling is the next selected for transmission.

Modeling of actual finish time, while feasible, is computationally intensive. The model needs to be substantially recomputed every time a packet is selected for transmission and every time a new packet arrives into any queue.

5. WFQ (Weighted Fair Queuing)

- Permite diferentierea fluxurilor prin indrumarea lor in cozi de asteptare diferite in raport cu clasa de trafic asociata. Ponderele determina rata cu care este servita clasa de trafic;
- Metoda foloseste campul IP precedence;
- Ponderile sunt stabilite de acest camp si nu pot fi modificate;
- Complexitatea planificarii depinde de numarul pachetelor in asteptare;
- Fluxurile care au campuri IP precedente setat vor obtine o rata de prioritate mai mare.

Exp. Daca avem 8 fluxuri cu ponderi de la 0 la 7, atunci traficul care are IP precedence setat in 0 va obtine:

$$IP_{precedence=0} = \frac{1}{1 + 2 + \dots + 8} = \frac{1}{36}$$

$$IP_{precedence=2} = \frac{3}{1 + 2 + \dots + 8} = \frac{3}{36} = \frac{1}{12}$$

6. CBWFQ (Class Based WFQ)

- Permite crearea a 256 clase de trafic, fiecare clasa avand propria coada de asteptare. Pentru fiecare clasa se garanteaza latimea de banda insa nu se garanteaza intarzierea.

CURS 6

Alte metode de planificare

1. Round Robin

Asigura organizarea in router a mai multor cozi de asteptare separate pentru fiecare linie de iesire. Routerul scaneaza aceste cozi pe rand, extragand pachetele pentru cozile care nu sunt goale. Un singur pachet poate fi extras intr-un ciclu.

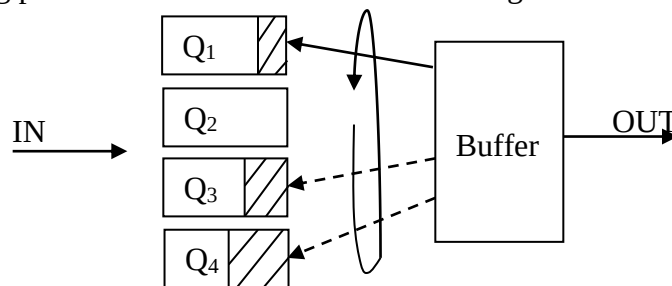
Daca unele cozi sunt goale, restul cozilor sunt interogate mai des.

Rata de servire a unei cozi depinde de numarul cozilor care au pachete cat de lungimea pachetelor.

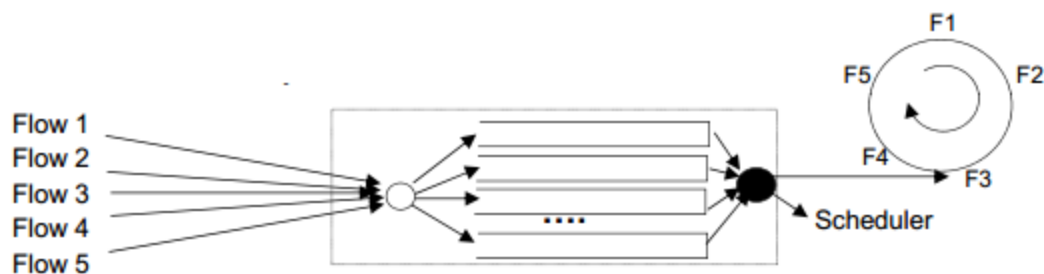
Processor sharing approximation. Buffer organized in separate queues, one queue for each active flow (Each queue is a FIFO queue).

- Service cycle among queues, one packet from each queue

Algoritmul permite unui router sa organizeze mai multe cozi de asteptare. Pachetele se extrag pe rand din fiecare coada care nu este goala.



Q_i = cozi de așteptare



Aceasta este metoda de utilizare a bufferilor hardware din placa de retea.

To improve delay fairness, at each serving cycle it is possible to modify queue service order

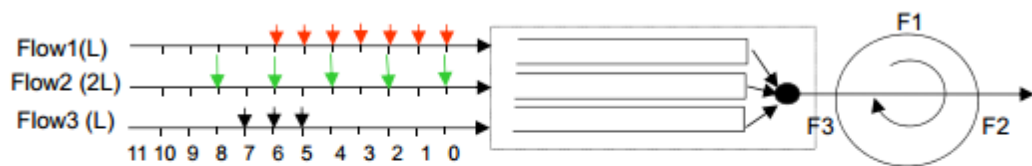
- At time 0, queue service order: 1,2,3,...,K
- At time 1, queue service order: 2,3,...,K,1

Easy to implement in hardware

- Guarantees flow isolation
 - Through queue separation
- Service rate of each queue:
 - C/K, for fixed packet size and k flows
 - For variable packet size, some unfairness may arise
 - Taking into account packet size makes implementation more complex

Dezavantajul acestui algoritm este ca acorda latime de banda mai mare echipamentelor care folosesc pachete de dimensiune mare. O solutie alternativa ce rezolva aceasta problema este prelucrarea datelor octet cu octet.

Round Robin: example



T	In F1	In F2	In F3	Q(F1)	Q(F2)	Q(F3)	Scheduled packets
0	P10[1]	P20[2]	-	P10	P20	-	F1:P10
1	P11[1]	-	-	P11	P20	-	F2:P20
2	P12[1]	P22[2]	-	P11,P12	P22	-	F2:P20 (cont)
3	P13[1]	-	-	P11,P12,P13	P22	-	F1:P11
4	P14[1]	P24[2]	-	P12,P13,P14	P22,P24	-	F2:P22
5	P15[1]	-	P35[1]	P12,P13,P14,P15	P24	P35	F2:P22 (cont)
6	P16[1]	P26[2]	P36[1]	P12,P13,P14,P15,P16	P24,P26	P35,P36	F3:P35
7	-	-	P37[1]	P12,P13,P14,P15,P16	P24,P26	P36,P37	F1:P12
8	-	P28[2]	-	P13,P14,P15,P16	P24,P26	P36,P37	F2:P24
9	-	-	-	P13,P14,P15,P16	P26,P28	P36,P37	F2:P24
10	-	P2A[2]	-	P13,P14,P15,P16	P26,P28	P36,P37	F3:P36

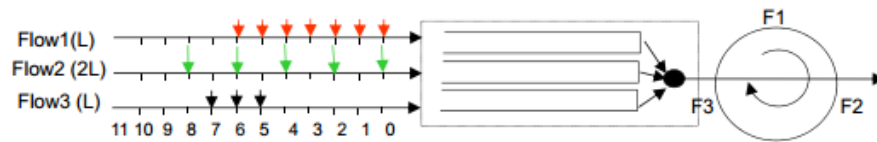
2. Deficit Round Robin

Acest algoritm foloseste o variabila pentru a determina numărul de biti dintr-o coada de asteptare care trebuiesc preluati intr-o runda. Daca fiecare coada de asteptare are pachete de transmis, iar lungimea pachetului este mai mica sau egala cu valoarea variabilei, atunci pachetul este transmis si se decrementează variabila cu lungimea pachetului.

Daca lungimea pachetului este mai mare decat valoarea variabilei se așteaptă runda urmatoare in care variabila este incrementata in mod automat.

Algoritmul poate defavoriza unele cozi de transmisie intr-o runda inasa acestea vor recupera in una din rundele urmatoare.

Deficit Round Robin: example



T	Inc.	D[1]	D[2]	D[3]	Q(F1)	Q(F2)	Q(F3)	Scheduled
0	F1	0+1-1	0	0	P10	P20	-	F1:P10
1	F2,F1	0+1-1	0+1	0	P11	P20	-	F1:P11
2	F2	0	1+1-2	0	P12	P22	-	F2:P20
3	-	0	0	0	P12,P13	P22	-	F2:P20(cont)
4	F1	0+1-1	0	0	P13,P14	P22,P24	-	F1:P12
5	F2,F3	0	0+1	0+1-1	P13,P14,P15	P22,P24	P35	F3:P35
6	F1	0+1-1	1	0	P14,P15,P16	P22,P24,P26	P36	F1:P13
7	F2	0	1+1-2	0	P14,P15,P16	P22,P24,P26	P36,P37	F2:P22
8	-	0	0	0	P14,P15,P16	P24,P26	P36,P37	F2:P22(cont)
9	F3	0	0	0+1-1	P14,P15,P16	P24,P26	P36,P37	F3:P36
10	F1	0+1-1	0	0	P15,P16	P24,P26	P37	F1:P14
11	F2,F3	0	0+1	0+1-1	P15,P16	P24,P26	P37	F3:P37
...								

- The **unweighted Deficit Round Robin (DRR) Scheduler**

- Information maintained by the **unweighted DRR scheduler**:

- **Deficit counter** = the **number of bytes** that a **flow** is **allowed to transmit** when it is its turn.

- **Quantum** = the **number of bytes** that is **added** to the **deficit counter** of flow in **each round**
(**Quantum** = **amount of credit per round**)

-
- **Operation of the DRR scheduler**:

- Packets from **flows** are **transmitted** in a **round robin** manner

- The **quantum** is **added** to the **deficit counter** of a flow **before servicing a flow**.

$$\text{deficit counter}(f) = \text{deficit counter}(f) + \text{quantum}$$

- A **packet** from a **flow** is **only transmitted** if:

- **deficit counter** $\geq$ **length of packet**

- If a **packet** is **transmitted**, the **deficit counter** of that **flow** is **updated** as follows:

$$\text{deficit counter}(f) = \text{deficit counter}(f) - (\text{\#bytes in packet})$$

Note:

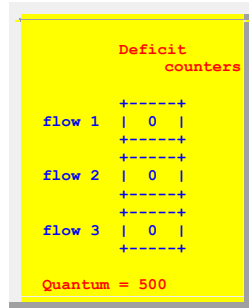
- When the **DDR router** detects the **first packet** of a **new flow**, it **allocates a deficit counter** for the **new flow** and **initialize** the **deficit counter** to **zero**

- **Example: unweighted Deficit Round Robin Scheduler**

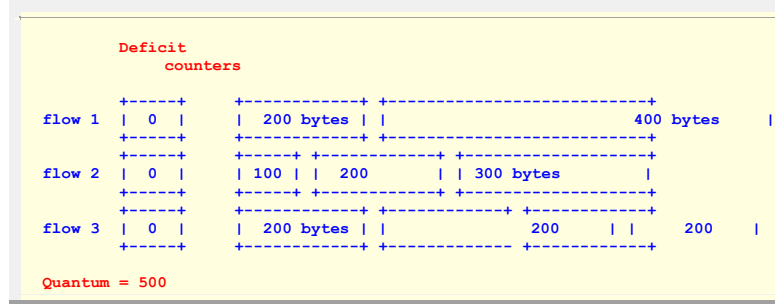
- **Example**:

- **3 flows**
 - **Quantum = 500 (bytes)**

Graphically:

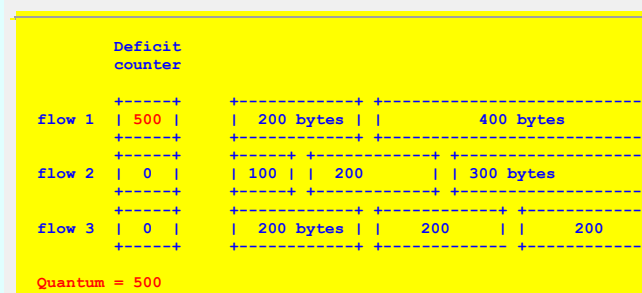


- Packet arrivals:

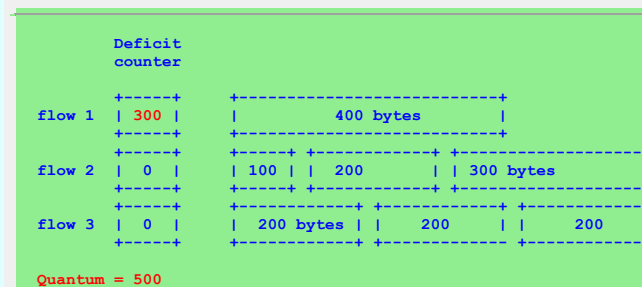


- Servicing Flow 1:

- Add quantum to flow 1's deficit counter:



- Transmit first packet from Flow 1:



Flow 1 does **not have enough credits** for the **next packet** and **ends its turn**
 Flow 1 will **carry** its **residual "transmission" credits** over to the **next service round**

- Servicing Flow 2:

- Add quantum to flow 2's deficit counter:



counter		
flow 1	300	400 bytes
flow 2	500	100 200 300 bytes
flow 3	0	200 bytes 200 200

Quantum = 500

- Transmit **first packet** from Flow 2:

Deficit counter		
flow 1	300	400 bytes
flow 2	400	200 300 bytes
flow 3	0	200 bytes 200 200

Quantum = 500

- Transmit the **second packet** from Flow 2:

Deficit counter		
flow 1	300	400 bytes
flow 2	200	300 bytes
flow 3	0	200 bytes 200 200

Quantum = 500

Flow 2 does **not have enough credits** for the **next packet** and **ends its turn**
Flow 2 will **carry** its **residual "transmission" credits** over to the **next service round**

- Servicing Flow 3:

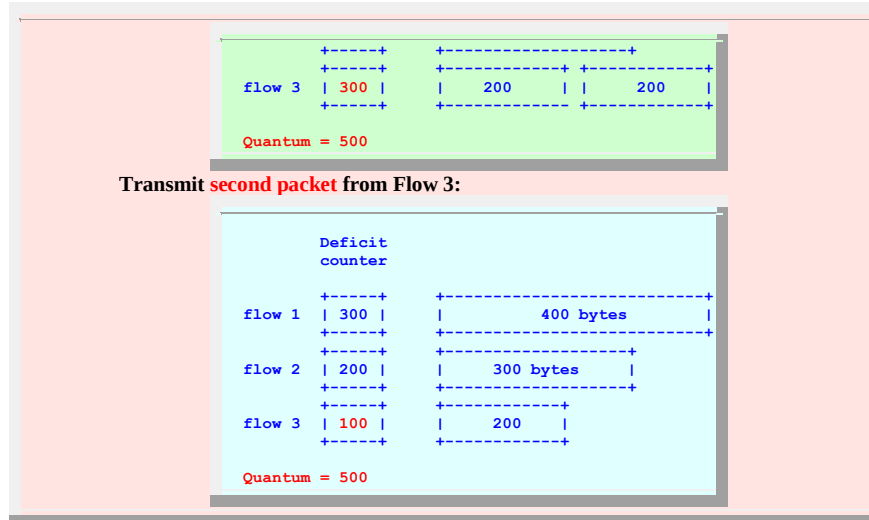
- Add **quantum** to flow 3's **deficiy counter**:

Deficit counter		
flow 1	300	400 bytes
flow 2	200	300 bytes
flow 3	500	200 bytes 200 200

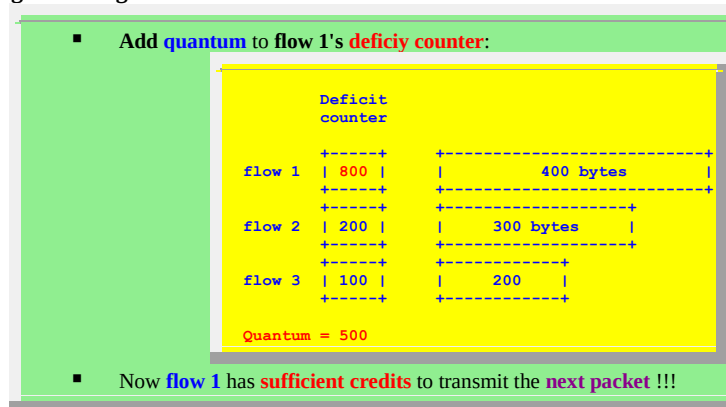
Quantum = 500

- Transmit **first packet** from Flow 3:

Deficit counter		
flow 1	300	400 bytes
flow 2	200	300 bytes



- Flow 3 does **not have enough credits** for the **next packet** and **ends its turn**
- Flow 3 will **carry** its **residual "transmission" credits** over to the **next service round**
-
- Servicing Flow 1 again:



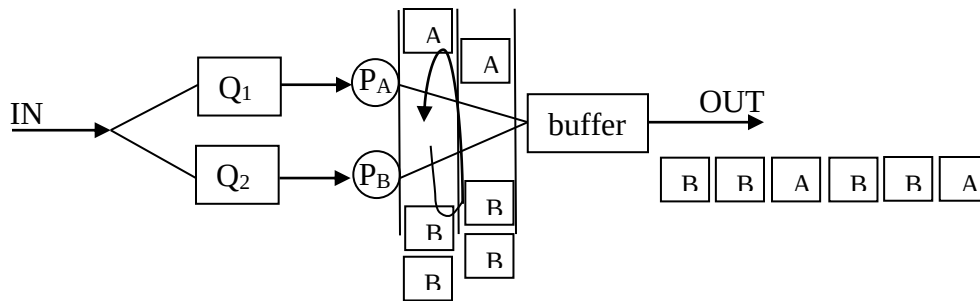
- And so on....
- From the operation, it should be clear that:
 - Flow will share the bandwidth fairly if they are **unsaturated**
 - The **saturated flows** (flows that transmit at a lower rate than the fair share) will receive its complete service rate. The left over transmission rate will be divided among the **unsaturated flows**

3. Weighted Round Robin

Aceasta metoda preia n pachete dintr-o coada de asteptare intr-un ciclu, cu n specific fiecărei cozi, valoarea n fiind aleasa astfel incat sa ocupe o anumita fractiune din latimea de banda.

Buffer partitioned in K queues and each queue served according to a FIFO discipline

Ex. Daca in router intra doua fluxuri A si B, din router sunt extrase



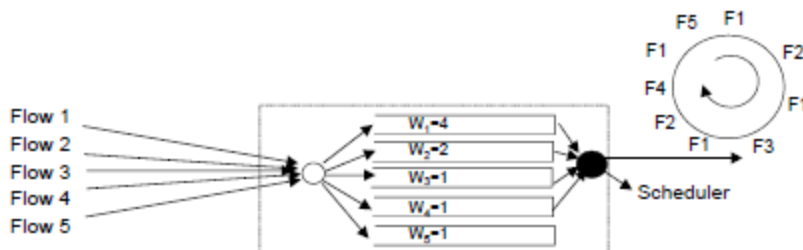
Acest algoritm lucrează bine când dimensiunea pachetelor este aceeași. Ciclurile tind sa devina lungi...

Exemplu: Pe o legatura de 45Mb/s, 500 de fluxuri au ponderea 1 si 500 de fluxuri au ponderea 10, dimensiune pachet 48octeti.

Timpul de serviciu pentru un pachet este de 9.422us

Un ciclu are nevoie de $500 + 500 \cdot 10 = 5500$ timpi de serviciu = 51.82ms

WRR: Weighted Round Robin



- If all flows are active
 - F1 obtains 4/9 of the link bit rate
 - F2 obtains 2/9
 - F3, F4 and F5 obtain 1/9

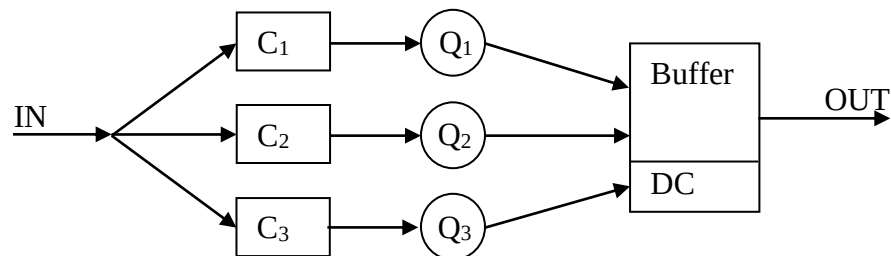
WRR: problems

- Service cycle (and fairness) may become long when
 - Many flows are active
 - Flows have very different weights
 - On a 45Mbit/s link, 500 flows with weight 1 and 500 flows with weight 10, cell size of 48byte
- Service time of one cell 9.422us
- A cycle requires $500 + 500 \cdot 10 = 5500$ service time = 51.82ms
- Service provided to flows may be bursty
 - Avoidable, but complex

- For each variation of the number of active flows (departure, arrival) service cycle must be redefined
 - How to deal with the remaining part of the cycle?
- To deal with variable packet size may use WDRR, Deficit Round-Robin extended to weight support
- May use weights in WRR to compensate for variable packet size for best effort traffic (requires knowledge of flow average packet size)

4. Deficit Weighted Round Robin

Fluxurile sunt clasificate in cozi de asteptare distincte dupa calitatea serviciului. Fiecare coada foloseste un indicator pentru numarul de octeti care vor fi preluati (Q) la un ciclu si un contor de deficit care cumuleaza intr-o variabila ce determina numarul total de octeti ce pot fi extrasi in runda curenta.



5. Planificarea prin eliminarea pachetelor

In schemele anterioare niciun pachet nu era pierdut. Prin aceasta metoda daca respectarea calitatii serviciului este pusa in pericol se ignora pachete din fiecare coada pentru a reduce intarzierea celorlalte pachete.

Pierderea pachetelor se face doar cu respectarea integritatii serviciului (se foloseste in cazul protocolului TCP mecanismul de evitare a congestiei).

Virtual Clock scheduling

- Time Division Multiplexing emulation
- Each flow j has an assigned service rate r_j
- To each data k of length L_j belonging to flow j , a tag (label, urgency, auxiliary virtual clock) is assigned
 - Tag represents the data finishing service time (starting service time + service time) in a TDM system serving flow j at rate r_j :

$$\text{Aux VC}_j^k = \text{Aux VC}_j^{k-1} + \frac{L_j}{r_j}$$

Virtual Clock scheduling

Example

	0	1	2	3	4	5	6	7	8	9
$r_1=1/3$	 3	 6	 9	 12	 15	 18				
$r_2=1/3$	 3	 6	 9	 12	 15	 18				
$r_3=1/3$	 3			 6			 9			 12

Service order: 
3 
3 
3 
6 
6 
6 
9 
9 
9

Virtual Clock: example 1



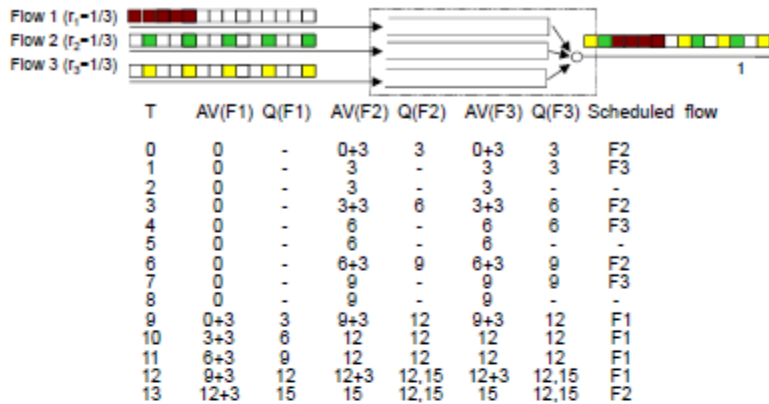
T	AV(F1) Q(F1)	AV(F2) Q(F2)	AV(F3) Q(F3)	Scheduled flow
0	0+3 3	0+3 3	0+3 3	F1
1	3+3 6	3+3 3,6	3 3	F3
2	6+3 6,9	6+3 3,6,9	3 -	F2
3	9+3 6,9,12	9+3 6,9,12	3+3 6	F1
4	12+3 9,12,15	12+3 6,9,12,15	6 6	F3
5	15+3 9,12,15,18	15+3 6,9,12,15,18	6 -	F2
6	18+3 9,12,15,18,21	18+3 9,12,15,18,21	6+3 9	F1
7	21+3 12,15,18,...	21+3 9,12,15,18,21	9 9	F3
8	24+3 12,15,18,...	24+3 9,12,15,18,...	9 -	F2
9				

Virtual Clock scheduling

Problem:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$r_1=1/3$	□ 3			□ 6			□ 9			□ 12			□ 15	
$r_2=1/3$	○ 3			○ 6			○ 9			○ 12			○ 15	
$r_3=1/3$										△ 3	△ 6	△ 9	△ 12	△ 15
	□ 3	○ 3		□ 6	○ 6		□ 9	○ 9		△ 3	△ 6	△ 9	□ 12	○ 12

Virtual Clock: problem



Virtual Clock

- Long term fairness with some problems
 - Inactive flows “gain time” and get more service in the future, penalizing, and even starving, other active flows (even conformant flows)
 - Clock of different flows proceed independently
- Modify the tag computation, taking into account system real time:

$$\text{Aux VC}_j^k = \max (\text{Aux VC}_j^{k-1}, a_j^k) + \frac{L_j^k}{r_j}$$

Virtual Clock scheduling

Problem solved:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$r_1=1/3$	□ 3			□ 6			□ 9			□ 12			□ 15	
$r_2=1/3$	○ 3			○ 6			○ 9			○ 12			○ 15	
$r_3=1/3$										△ 12	△ 15	△ 18	△ 21	△ 24
	□ 3	○ 3		□ 6	○ 6		□ 9	○ 9		□ 12	○ 12	△ 12	□ 15	○ 15

Modified Virtual Clock

Another problem:

Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$r_1=1/3$	□ 3	□ 6	□ 9	□ 12	□ 15	□ 18	□ 21	□ 24	□ 27	□ 30	□ 33	□ 36	□ 39	
$r_2=1/3$	○ 3	○ 6	○ 9	○ 12	○ 15	○ 18	○ 21	○ 24	○ 27	○ 30	○ 33	○ 36	○ 39	
$r_3=1/3$													△ 15	△ 18
	□ 3	○ 3	□ 6	○ 6	□ 9	○ 9	□ 12	○ 12	□ 15	○ 15	□ 18	○ 18	△ 15	△ 18

<http://www.mathcs.emory.edu/~cheung/Courses/558/Syllabus/11-Fairness/DRR.html>

Virtualizarea

Este unul dintre subiectele cele mai discutate astăzi în IT. A fost posibilă datorită creșterii în viteză și capacitățile hardwareului comun. Oferă o modalitate atractivă de exploatare la maxim a hardwareului. Termenul de “virtualizare” în literatura de piață rivalizează cu termene precum “Internet” sau “rețea” în anii ‘90. „ Virtualizarea este întâlnită cel mai des în rețelistică, sisteme de stocare, procese server, la nivel de sistem de operare și la nivel de mașină

Definition Virtualization is an abstraction of computer resources. We can access resources in a consistent way before and after abstraction through virtualization. This kind of resource abstraction is not limited by implementation, geographical location or the underlying physical configuration

Oferă separarea logică a cererii pentru anumite servicii de resursele fizice care oferă acele servicii

Oferă un nivel logic de abstractizare care eliberează aplicațiile, serviciile de sistem și chiar sistemul de operare de a lega cu o anumită bucată de hardware

Oferă abilitatea de a rula aplicații, sisteme de operare, sau servicii de sistem într-un mediu de sistem distinct logic care este independent de sistemul de calcul fizic

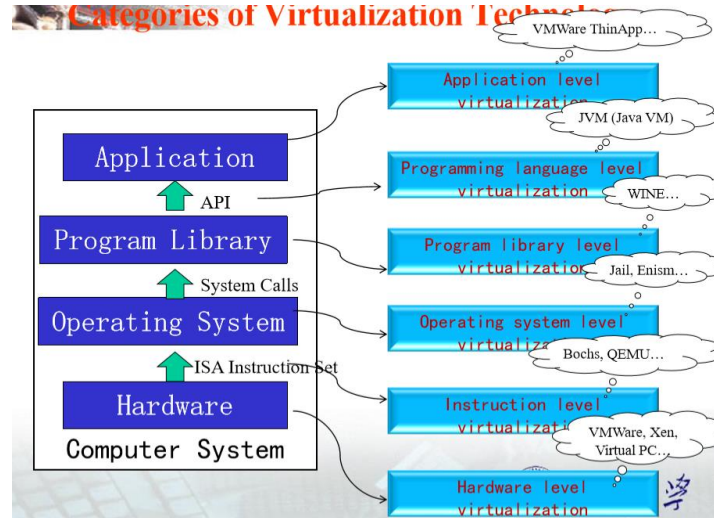
Exemplul clasic (cunoscut) : memoria virtuală

- Permite unui sistem de calcul să apară ca având mai multă memorie decât cea fizică instalată în acel sistem
- Este o tehnică de management a memoriei care permite unui sistem de operare să vadă segmente de memorie necontinue ca și un singur spațiu continuu de memorie
- Este implementată în mod tradițional în sistemele de operare prin paginare care permite sistemului de operare să utilizeze un fișier sau o porțiune a unui dispozitiv de stocare să salveze pagini de memorie care nu sunt în utilizare activă
- Cunoscut ca și un “fișier de paginare” sau “spațiu swap”, sistemul poate transfera foarte repede pagini de memorie la și de la această arie după cum SO sau aplicații în rulare cer accesul la conținutul acestor pagini
- SO precum Unix (incluzând Linux, *BSD OS, Mac OS X) și Microsoft Windows utilizează o formă de mașină virtuală (VM) pentru a permite SO și aplicațiilor să acceseze mai multe date decât este posibil să încapă în memoria fizică

Pentru calculatoare virtualizarea constă în crearea unei versiuni software de platformă hardware, S.O., rețele sau stocare. Conceptul de virtualizare a apărut din mai multe motive:

- Ușurarea sarcinilor administrative;
- Îmbunătățirea scalabilității;
- Din motive economice.
- Din motive de spațiu

Common types of Virtualization



□ Types:

Infrastructure Virtualization,

System Virtualization,

Software Virtualization.

□ Infrastructure Virtualization

□ Network Virtualization: Integrate network hardware resources with software resources to provide users with virtualization technology of virtual network connection. It can be divided into VLAN and VPN.

□ Storage Virtualization: Provide an abstract logical view of physical storage device, so the user can access the integrated storage resources through unified logical interface of this view. It can be divided into **storage device based storage virtualization**(eg RAID) and **network based storage virtualization**(eg NAS, SAN).

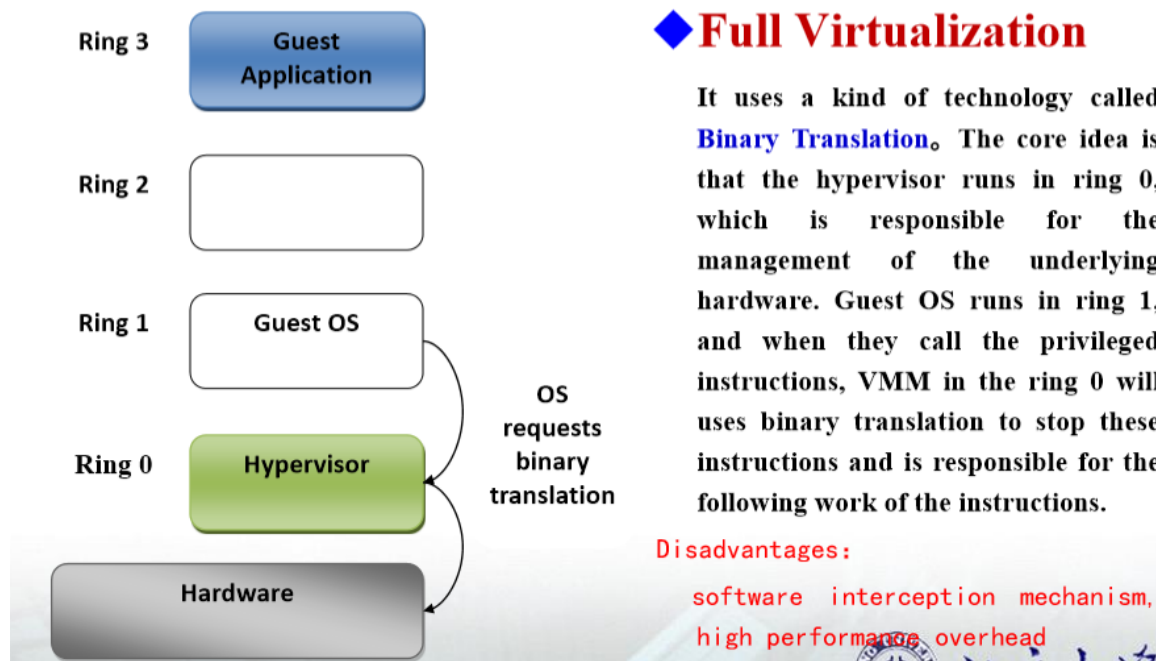
□ System Virtualization

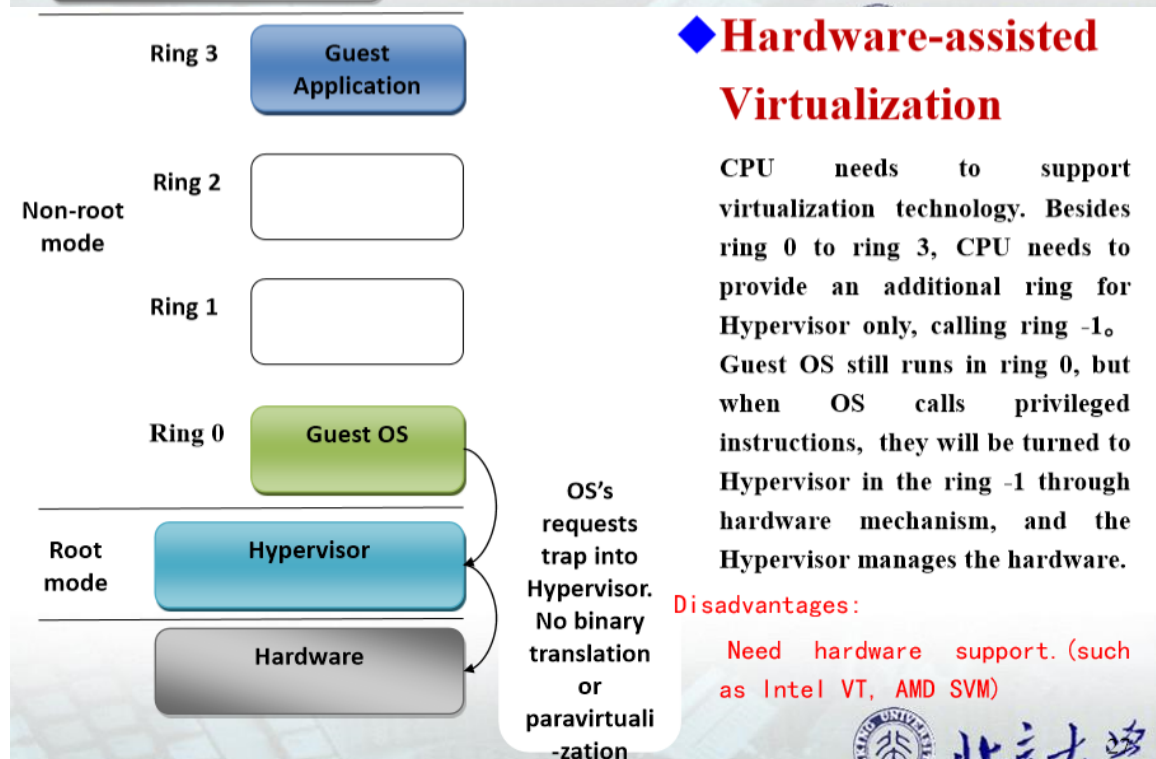
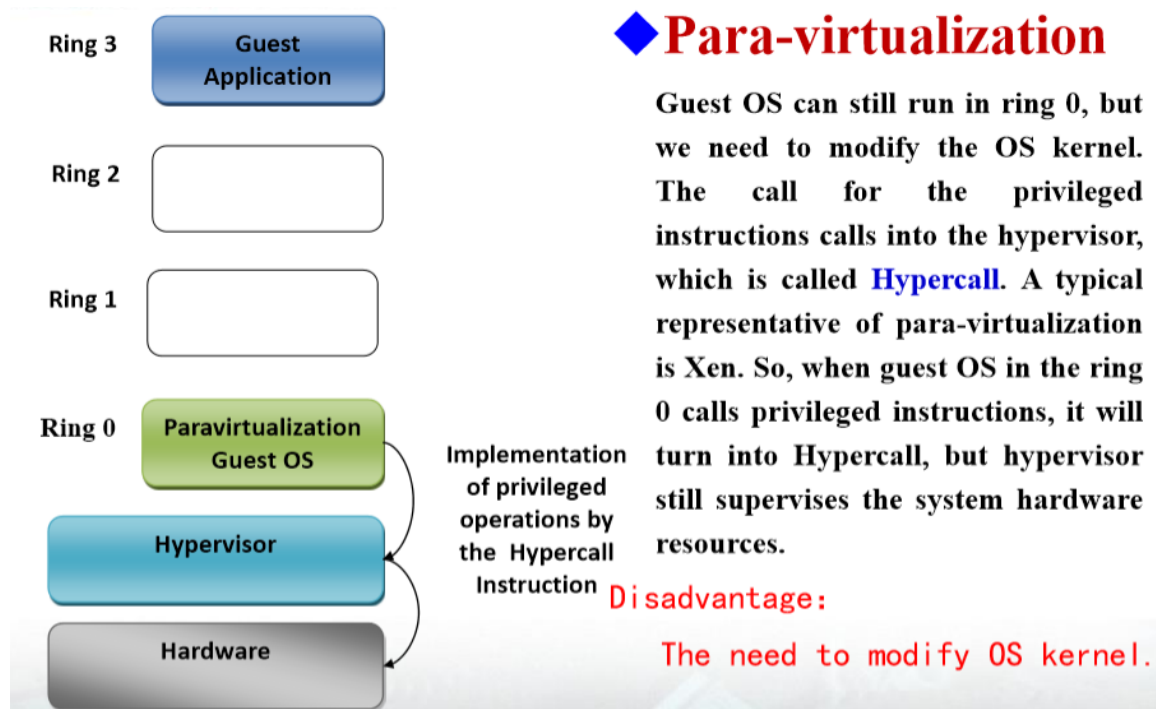
□ Core idea : Create one or more virtual machines using virtualization software on physical machine. □ PC/Server Virtualization : The maximum value of system virtualization. □ Desktop Virtualization : Solve the coupling relationship between PC desktop environment(including applications and files, etc.) and physical machines. Virtualized desktop environment is stored on a remote server, and when user has compatible device with sufficient display ability(eg PC, Smart Phones, etc.), all the programs and data will eventually stored in the remote server.

□ Software Virtualization

□ The High-level language virtualization : Solve the migration problem of executable programs between different architectures. Programs which are written in high-level language will be compiled into standard intermediate instructions, and these instructions will be executed during interpretation or compiled environment(such as Java virtual machine JVM) □ Application Virtualization : Decouple applications from operation systems, and provide a virtual running environment for applications, including application executable files and required runtime environment. Application virtualization server can push user required program components to the client virtual running environment timely(such as VMWare ThinApp).

Classification of Virtualization implementation technologies





- Nivele de virtualizare
1. Virtualizarea aplicatiei
 2. Virtualizarea desktopului
 3. Virtualizarea retelei
 4. Virtualizarea hardware (a serverului sau masinii)
 5. Virtualizarea memoriei

6. Virtualizarea stocarii
7. Virtualizarea la nivel de sistem sau a sistemului de operare

1.Virtualizarea aplicatiei

„ Termenul descrie procesul de compliare a aplicatiilor in code binar independent de masina care sunt executate in orice sistem care ofera o masina virtuala adecvate ca mediu de executie

„ Aplicatii compilate in cod binar devin entitati logice care sunt executate in sisteme fizice diferite cu diferite caracteristic legate de sistemui de operare sau arhitectura procesor

„ Exemple ale acestei abordari:

 %oCel mai cunoscut: codul produs de compilatoarele pentru limbajul de programare Java

 %oConceptul a fost introdus in 1970 prin UCSD P cu cel mai popular compilator fiind UCSD Pascal compiler.

 %oMicrosoft a adoptat o abordare similara in Common Language Runtime (CLR) utilizata cu aplicatiile .NET, „ Codul scris in limbajele care suporta CLR sunt trasnformate, la timpul compilarii, in CIL (Common Intermediate Language), cunoscut anterior ca MSIL (Microsoft Intermediate Language). „ CIL ofera un set de instructiuni independente de platforma care pot fi executate in orice mediu ce suporta .NET Framework.

2. Virtualizarea desktopului

„ Termenul descrie abilitatea de a afisa un display grafic de la un sistem de calcul la un alt sistem de calcul sau dispozitiv de afisare inteligent

„ Exemple:

 %oVirtual Network Computing,

 %oClienti precum Microsoft Remote Desktop si produsele Terminal Server

 %oServere terminal Linux precum Linux Terminal Server

 %oSistemul X Window System si protocolul de administrare a fesestrelor XDMCP.

„ Suport intern pentru desktopuri virtuale multiple intre care utilizatorul poate naviga.

„ Supporta si virtualizare la nivel de ecran sau display permitand mangementul de ferestre sa utilizeze o regiune de display care este mai mare decat dimensiunea fizica a monitorului

„ Este un exces ca sa fie numita “virtualizare” nefind un exemplu clasic de virtualizare!

 %oRealizeza, dintr-o consola grafica a oricarui sistem suportat, o entitate logica care este accesata si utilizara pe diferite sisteme fizice de calcul

 %oConsola la distanta, sistemul de operare in care ruleaza, si aplicatiile sunt executate pe o masina fizica specifica.

 %oNumirea software-ul de afisare la distanta o tehnologie de virtualizare este echivalenta cu a considera ca un telescop este o multime de ochi virtuali cu ajutorul caruia se poate vedea mult mai departe

3.Virtualizarea rețelei

„ Termenul descrie abilitatea de referire a resurselor de rețea în mod logic în locul specificării dispozitivelor fizice, configurațiilor sau colecțiilor de mașini specifice.

„ Nivele

  %Virtualizare dispozitivelor de rețea a mașinilor singulare care permite mașinilor virtuale multiple să partajeze o singură resursă de rețea fizică,

  %Concepte la nivel enterprise precum rețele private virtuale sau tehnologii de rutare pentru crearea de subrețele și segmentarea rețelelor existente

„ Exemple:

  %Xen se bazează în virtualizarea rețelelor virtualization prin pachetul Linux bridge - utils care permite mașinilor virtuale să apară ca având o adresă fizică unică (adrese Media Access Control, sau MAC) și adrese IP unice.

  %Soluciile de virtualizare a serverului, precum UML, utilizează dispozitive de rețea Linux virtual Point - to - Point (TUN) & Ethernet (TAP) pentru a oferi acces la spațiul utilizatorului prin rețeaua gazdei.

  %Switchuri su rutere avansate de rețea utilizează tehnici precum Virtual Routing & Forwarding (VRF), VRF - Lite, și Multi - VRF pentru segregarea traficului în segmente de rețea pentru a permite domenii de rutare virtuale multiple într-o singură piesă hardware de rețea

Virtualizarea serverului sau mașinii (hardware)

„ Termenul descrie abilitatea de a rula o întreaga mașină virtuală, incluzând sistemul sau de operare, pe alta mașină

„ Fiecare mașină virtuală care rulează pe un sistem de operare părinte

  „ Este logic distinctă,

  „ Are acces la anumite sau tot hardwareul sistemului gazdă,

  „ Are asignările sale logice pentru dispozitivele de stocare pe care SO este instalat,

  „ Poate rula propriile aplicații în propriul mediu de operare

  „ Virtualizarea serverului: tipul de virtualizare la care se gândește majoritatea lumii când se referă la “virtualizare”!

„ Nu la fel de comun, termenul de “virtualizare a mașinii” identifică în mod unic acest tip de virtualizare,

  %Diferențiază în mod clar nivelul la care are loc virtualizarea — mașina în sine este virtualizată — indiferent de tehnologia care este utilizată.

„ Exemple: „ KVM,Microsoft Virtual Server&Virtual PC,Parallels Workstation,User Mode Linux,Virtual Iron, „ VMware, Xen.

„ Virtualizarea serverului este în mod uzual diferită de termenul “server virtual”

  %Serverul virtual este utilizat pentru a descrie următoarele: „

Capabilitatea serverelor e - mail sau Web să servească domenii Internet multiple

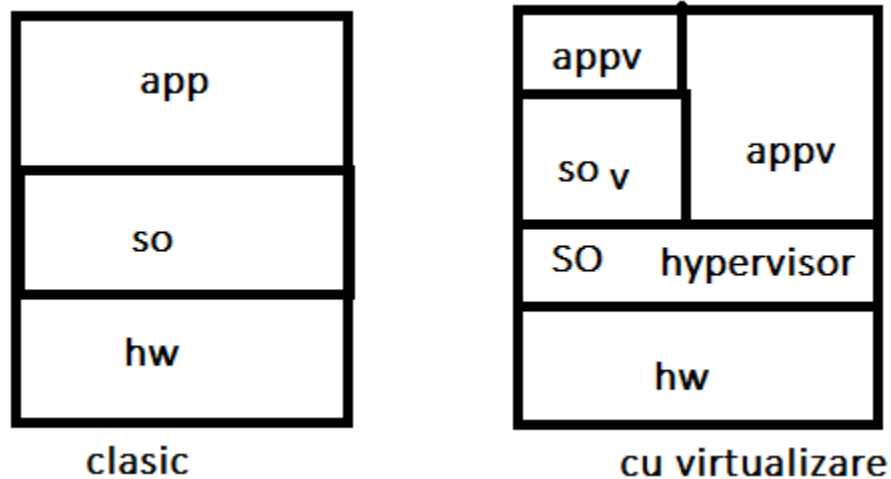
„ Tehnologii de virt. la nivel sistem pentru a oferi utilizatorilor unui ISP o mașină server virtuală.

Utilizarea unei mașini virtuale:

Aspect cheie: masinile virtuale diferite nu partajeaza acelasi nucleu si pot de aceea rula in diferite sisteme de operare. „ Differa de virtualizarea la nivel sistem in care serverele virtuale partajeaza un singur nucleu „ Oferă o serie de oportunitati pt.infrastructura, client &business: %oRularea de legacy-software, cand se depinde de produsul care ruleaza numai pe o versiune specifica a sistemului de operare %oMedi de testare a sistemelor software si de asigurare a calitatii, cand este necesara testarea unui singur produs pe mai multe sisteme de operare diferite si pe mai multe versiuni ale unui sist de operare %oMedii de dezvoltare la nivel de baza, cand dezvoltatorii trebuie sa lucreze cu anumite versiuni de unelte, nucleu de sistem de operare, sau o distributie specifica de sistem de operare.

Abordari pentru virtualizarea serverului/masinii

1.SO gazda: „ Fiecare server ruleaza intr-o instanta de sistem de operare separata intr-o aplicatie de virtualizarea care ea insusi ruleaza intr-o instanta a unui sistem de operare specific „ Examples: Parallels Workstation, VMWare Workstation, and VMWare GSX Server „ SO-ul in care aplicatia de virtualizare ruleaza este referita ca un “SO gazda”.



2. Parallel Virtual Machine: „ Sisteme fizice sau virtuale sunt organizate intr-o masina virtuala utilizand software de grupare precum PVM. „ Gruparea este capabila sa efectueze calcule complexe intensive in calcul sau date „ Acesta este mai mult un concept de grupare (clustering) decat o solutie de virtualizare

3. Bazata pe hipervizor:

- Un monitor mic de masini virtuale – hypervisor – ruleaza pe hardwareul masinii oferind:
 1. identifica, intrerupe, raspunde la operatiile protejate sau privilegiate ale CPUului efectuate de fiecare VM
 2. trateaza cozile, distribuie si returnarea rezultatelor cererilor hardware de la VMuri
- Un SO administrativ ruleaza peste hipervizor, precum fac si masinile virtuale
 - Poate comunica cu hipervizorul
 - Utilizat pentru administrarea instantelor de masini virtuale
- Exemplu: conceptul de paravirtualizare
 - Este modelul primar utilizat de Xen,
 - Foloseste un nucleu Linux special pentru a suporta mediul sau de administrare numit domain0.
 - Ruleaza versiuni nemodificate de SOuri peste hypervisor.

4. Virtualizarea completa:

- similara paravirtualization,
- Utilizeaza un hipervizor, dar incorporeaza cod in hipervizorul care emuleaza hardwareul
- Exemplu: VMWare ESX server utilizeaza o versiune adaptata de Linux (cunoscuta ca si Service Console) pentru SOuri sau administrativ.

5. Virtualizare la nivel de nucleu:

- Nu necesita un hypervisor,
- Ruleaza o versiune separata de nucleu Linux & o VM asociata ca proces al spatiului utilizator pe masina fizica
- Exemple:
 - User - Mode Linux (UML),
 - Necesita o constructie speciala a nucleului de Linux pentru sistemele de operare gazda
 - Kernel Virtual Machine (KVM),
 - Utilizeaza un driver de dispozitiv in nucleul gazda pentru comunicare intre nucleul Linux si VMuri
 - Necesita suport procesor pentru virtualizare (Intel VT sau AMD – v Pacifica),
 - Utilizeaza un proces QEMU modificat ca si cotainer pentru executie si afisarea VMurilor

6. Virtualizare hardware:

- Similar cu paravirtualizarea si virtualizarea completa: utilizeaza un hipervizor
- Disponibila numai in sistemele care ofera suport hardware pentru virtualizare:
 - Ultima generatie de procesoare de la Intel (Intel VT, aka Vanderpool) si AMD (AMD - V, aka Pacifica)
- Exemple de tehnol. Care pot sa profite de suportul hardware:
 - Sistemele bazate pe hypervisor precum Xen si VMWare ESX Server,
 - Tehnologiile la nivel de nucleu precum KVM,
- VMurile pot rula SOuri nemodificate

\Concluzii:

- Virtualizarea bazate pe hipervizor este cea mai populara tehnica de virtualizare astazi!
 - IBM's VM operating system, VMWare's ESX Server, Parallels Workstation, Virtual Iron products, and Xen.
- Istoric, utilizarea unui hipervizor:
 - Inceputa prin mediul comercial bazat pe masina virtuala IBM's CP/CMS OS (1966),
 - Popularizat de IBM ' s VM/370 SO introdus in 1970.
 - ...
 - 2006: VMware propune Virtual Machine Interface (VMI) generica care poate tehnologiilor de virtualizare bazate pe hipervizor multiple sa utilizeze o interfata la nivel de nucleu comuna
 - VMware si Xen au cazut de acord sa lucreze la dezvoltarea unui interfete generice numite paravirt_ops, care este in dezvoltare de catre IBM, VMware, Red Hat, si XenSource.
 - Includerea paravirt_ops intr-un nucleu permite oricarei tehnologii de virtualizare bazata pe hipervizor sa lucreze cu un nucleu Linux de baza
 - Proiecte nucleu precum cel al KVM permit utilizatorilor sa ruleze VMuri intr-un alt SO pe baza unui hardware adecvat fara a fi necesar hipervizorul

5. Virtualizarea memoriei

permite unui calculator sa utilizeze resursele virtuale dintr-un centru de date drept memorie virtuala; pentru utilizarea eficienta in majoritatea cazurilor calculatorul este un cluster.

Memoria este accesata de SO si/sau aplicatii in plus fata de memoria SO-ului local. Rolul ei poate fi de stocare, transmitere mesaje sau partajare date intre CPU si GPU (graphic processing unit).

Exista doua forme de implementare:

- la nivel de aplicatie in care accesarea memoriei virtuale se face printr-un API dedicat
- un sistem de fisier si integrarea SO caz in care se creeaza spatii de memorie puse la dispozitia aplicatiilor.

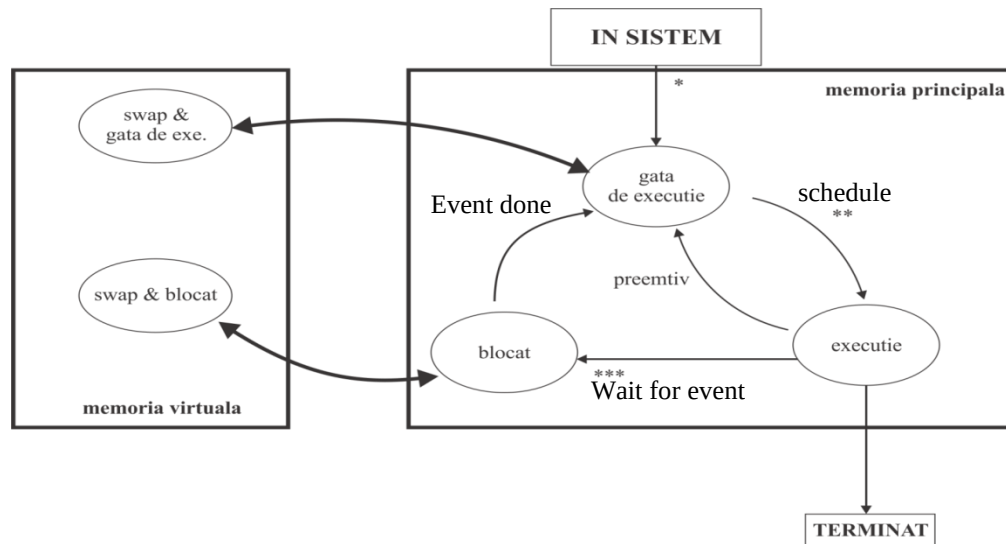
Memoria virtuala:

- Permite programelor rulanad in kernel-uri multi task sa acceseze diferite tipuri de memorii in mod unitar;
- Cea mai vizibila e memoria virtuala a WIN ce poate fi repartizata atat pe hard disk cat si pe discuri flash sau alte medii de stocare;
- Ascunde fragmentarea, realocarea memoriei si adresarea relativa.

Stările procesului: un proces poate sa aiba 3 stări: Gata de executie ;Execuție; Blocat.

Acestea 3 daca ne aflam in memoria principala si inca 2 stari in memoria virtuala:

- SWAP si gata de executie
- SWAP si blocat



Memory virtualization allows networked, and therefore distributed, servers to share a pool of memory to overcome physical memory limitations, a common bottleneck in software performance. With this capability integrated into the network, applications can take advantage of a very large amount of memory to improve overall performance, system utilization, increase memory usage efficiency, and enable new use cases. Software on the memory pool nodes (servers) allows nodes to connect to the memory pool to contribute memory, and store and retrieve data. Management software and the technologies of [memory overcommitment](#) manage shared memory, data insertion, eviction and provisioning policies, data assignment to contributing nodes, and handles requests from client nodes. The memory pool may be accessed at the application level or operating system level. At the application level, the pool is accessed through an API or as a networked file system to create a high-speed shared memory cache. At the operating system level, a page cache can utilize the pool as a very large memory resource that is much faster than local or networked storage.

Memory virtualization implementations are distinguished from [shared memory](#) systems. Shared memory systems do not permit abstraction of memory resources, thus requiring implementation with a single operating system instance (i.e. not within a clustered application environment).

Memory virtualization is also different from storage based on flash memory such as [solid-state drives](#) (SSDs) - SSDs and other similar technologies replace hard-drives (networked or otherwise), while memory virtualization replaces or complements traditional RAM.

1. Virtualizarea stocării datelor

Este un concept similar cu virtualizarea memoriei deoarece are rolul de a abstractiza (separa) stocarea locala de resursele fizice. Ea permite extinderea definiției de sistem de stocare de la echipamentul de stocare situat local la cel care folosește (de obicei) rețeaua locala; Permite o cantitate mai mare de stocare fizica sa fie disponibila sistemelor individuale. Permite sistemelor de fisiere existente sa creasca independent de problemele legate de legatiru simbolice si puncte de montare a dispozitivelor din retea.

Caracteristici ale virtualizării stocării:

- Realocarea spațiului de adrese: sistemul de virtualizare poate realoca adresele fizice utilizate de anumite date;
- Folosirea metadatelor: aceste informații folosesc pentru a menține o vedere de ansamblu asupra dispozitivului virtualizat;

- Redirecționarea cererilor de intrare-iesire: sistemele de virtualizare redirecționează cererile de intrare-iesire din spațiul virtual în cel real;
- Duplicarea (copierea) spațiului de stocare: această funcție permite copierea dispozitivelor logice. Este una din funcțiile cel mai greu de implementat;
- Managementul spațiului de stocare: asigură crearea, stergerea și modificarea dispozitivului de stocare virtuală. De exemplu, mărirea spațiului de stocare poate necesita un nou echipament de stocare.

Problemele apar datorită lipsei compatibilității, realizarea unui back up incorect, complexitatea ridicată și scalabilitatea redusă.

Modalitatea de implementare:

- Implementarea la nivel de host implică rularea unui soft adițional în majoritatea cazurilor integrat cu SO;
- La nivelul de dispozitiv de stocare: **aceasta asigurată în hard în cazul dispozitivelor write**. Această metodă de implementare asigură funcții precum clonarea și realizarea back-upului;
- La nivelul de rețea implică folosirea unui server care gestionează o rețea de mare viteză numită SAN (Storage Area Network).

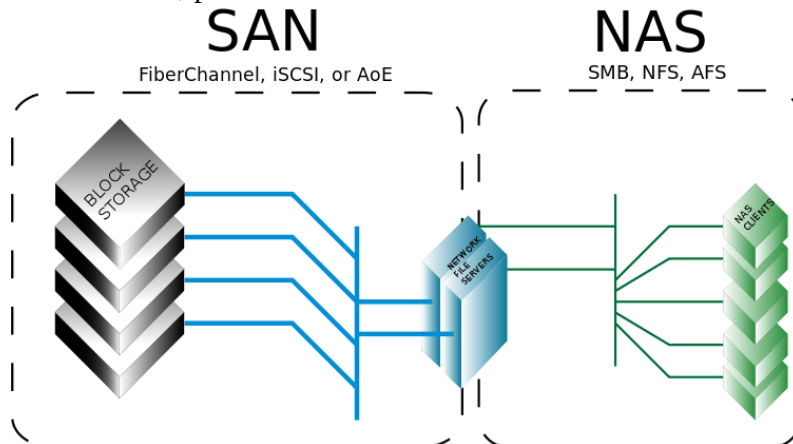
Există două moduri de virtualizare:

- o **virtualizarea la nivel de bloc** → se referă la separarea între spațiul de stocare local și spațiul fizic de stocare astfel încât să poată fi accesat în mod unitar deși spațiul de stocare fizic poate fi format din mai multe componente eterogene. Protocoalele care folosesc această metodă sunt: Fibre Channel, iSCSI (Internet Small Computer System Interface), SAS (Serial attached SCSI).

SAN(Storage Area Network) asigură stocare la nivel de bloc care lasă alegerea și organizarea sistemului de fișiere la latitudinea clientului.

Protocoalele SAN includ SCSI, Fibre Channel, iSCSI, ATA over Ethernet (AoE), sau HyperSCSI.

Spațiul disponibil prin rețeaua SAN apare pentru sistemul de operare al clientului ca o unitate de stocare, vizibilă în utilitățile de gestionare a spațiilor de stocare alături de unitățile de disc locale, putând fi formatat și montat în mod obișnuit.



- **Virtualizarea fisierelor** → prezenta in special in dispozitivele de stocare **NAS (Network Attached Storage)** ce elimina dependenta intre datele accesate la nivel de fisier si locatia fizica a acestora. NAS apare pentru sistemul de operare al clientului ca un file server care poate fi mapat de client ca un network drive. De obicei este folosit protocolul pentru sisteme distribuite de fisiere NFS (*Network File System*); Performanta acestor sisteme depinde de viteza retelei. Nodul central (cel care gestionează) nu trebuie sa fie foarte performant. Avantajele acestei solutii sunt:
 - Folosirea protoacoalelor de comunicatie pentru schimbul de date;
 - Folosirea listelor de acces pentru utilizatori;
 - Usureaza sarcini precum copierea si toleranta la erori.

NAS (Network Attached Storage)

Este o solutie pentru stocarea datelor cu accesare la nivel de fisier conectate la o retea de calculatoare, asigurand acces la date intru-un grup eterogen de clienti. NAS asigura atat stocarea cat si sistemul de fisiere.

[FreeNAS](#), OpenMediaVault si [Openfiler](#) sunt sisteme de operare pentru solutii NAS.

Sistemele NAS au un consum redus de energie, iar viteza de transfer variaza in functie de implementare intre 40MB/s si 120MB/s.

NAS este un mai degrabă calculator specializat, construit pentru stocarea fișierelor și servire - decât pur și simplu un calculator de uz general fiind utilizate pentru rol. Sistemele NAS sunt interconectate aparatele care conțin unul sau mai multe hard disk-uri, de multe ori aranjate în recipiente de depozitare logice, redundante sau RAID. Unitățile NAS, de obicei, nu au o tastatură sau ecran, și sunt controlate și configurate în rețea, folosind adesea un browser.

Alternative la NAS:

- Direct-Attached Storage (DAS): este o extensie a unui server care nu implica in mod obligatiu retelele de calculatoare;
- Clustered NAS este un NAS care se utilizează un sistem de fișiere distribuit care rulează simultan pe mai multe servere. Diferența esențială dintre un NAS cluster și tradițional este abilitatea de a distribui datele și metadatele peste nodurile cluster sau dispozitive de stocare. Cluster NAS, cum ar fi una tradițională, încă mai oferă acces unificat la fișierele din oricare din nodurile clusterului, nu au legătură cu localizarea efectivă a datelor.

NFS (Network File System);

- este un protocol care permite accesarea fisierelor prin intermediul retelelor de calculatoare;
- este posibila accesarea de catre mai multi utilizatori in paralel a unuia sau mai multor fisiere.
- **NFS** elimină responsabilitatea de distribuire a fișierelor de catre alte servere din rețea. **NFS** oferă de obicei acces la fișiere utilizând protocole de rețea de partajare de fișiere, cum ar fi NFS (*Network_File_System*), SMB (*Server Message Block* - SO Windows) sau AFP (*Apple*).

În 2009, vânzătorii NAS (în special la rețelele CTERA și NETGEAR) a început să introducă soluții Online Backup integrate în aparatele lor NAS, pentru recuperare caz de dezastru on-line.

Aplicatii pentru managementul stocarii

- Sistemele pentru managementul stocarii ruleaza de obicei intr-o masina virtuala;
- Sunt portabile deoarece masinile virtuale pot fi instalate pentru orice suport de SO;
- Sistemele de management trebuie sa asigure compatibilitatea intre domenii diferite de stocare si sa extinda facilitatile acestor dispozitive;
- Exista mai multe tipuri de dispozitive de management in functie de pret si performanta dorita;
- Avantajul acestor sisteme soft este posibilitatea update-ului acestora in timp.

Virtualizare sistem de operare

- Termenul descrie numeroase implementari pentru rularea unor medii de sistem multiple si logic distincte intr-o singura instanta a unui sistem de operare
- Bazat pe conceptul chroot (chroot) care este disponibil in toate sistemele tip UNIX
 - La bootarea sistemului, nucleul poate utiliza sisteme de fisiere root precum cele care sunt oferite de discurile RAM initiale si pentru a incarca driverele si a efectua sarcini primare de initializare a sistemului
- Mecanismul chroot este extins in virtualizarea la nivel de sistem:
 - Permite sistemului sa poarte servere virtuale cu propria multime de procese care se executa relativ la propriul sistem de fisiere
- Operarea pe propriile directoare
 - Previne ca serverele virtuale sa fie capabile sa acceseze fisierele altora
 - Daca un server este compromis, are acces numai la fisierele sale
- Diferentiatorul de baza intre virtualizarea serverului si cea la nivel de sistem este data de posibilitatea de rularea de diferite sisteme de operare in diferite sisteme virtuale
 - Daca toate serverele virtuale trebuie sa partajeze o singura copie a unui nucleu a unui sistem de operare => virtualizare la nivel de sistem
 - Daca servere virtuale diferite pot rula in sisteme de operare diferite si versiuni diferite ale unui singur sistem de operare => virtualizare server

- Exemple:
 - FreeBSD – chroot; Linux VServer, FreeVPS, and OpenVZ; Solaris Zones and Containers; Virtuozzo
- Avantaje fata de virtualizarea serverului/masinii:
 - Datorita partajarii unei singure instante a unui nucleu SO, virt. la nivel sistem este semnificativ mai simpla decat virt. serverului
 - Permite ca o singura gazda sa suporte mai multe “servere virtuale” decat numarul de VMuri care-l poate suporta
 - FreeBSD’s chroot jails, Linux - VServer, si FreeVPS sunt utilizate de mai multi ani in businesses precum ISPuri
 - Oferă fiecarui utilizator propriul server virtual in care are control complet fara compromiterea securitatii sistemului
 - Utilizata pentru *consolidarea serverului* : rularea de servere virtuale multiple pe un singur sistem hardware
- Principalul dezavantaj:
 - O problema de nucleu sau driver poate conduce la caderea tuturor serverelor virtuale suportate de acest sistem

Virtualizarea datelor

~~Virtualizarea datelor privește reprezentarea datelor independent de modul de stocare al acestora. Virtualizarea bazelor de date privește separarea nivelului aplicație de nivelul stocare.~~

~~Partitionarea bazelor de date este necesara datorita traficului sustinut, generat in retea. Cautari simultane efectuate intr-o singura baza de date duce la cresterea latentei.~~

~~Separarea componentelor bazelor de date se poate realiza in doua moduri:~~

- ~~— **Shared all database** — datele sunt repartizate pe calculatoare diferite inasa este permisa comunicarea intre nodurile rețelei pentru a mentine sincronizarea datelor. Aceasta tehnica este folosita in cadrul clusterelor de dimensiuni mici;~~
- ~~— **Shared nothing** — in care sistemele independente de baze de date sunt gestionate in mod individual. Nu toate bazele de date se preteaza pentru o astfel de implementare. Aceste sisteme necesita partajare manuala~~

~~Performanta acestor sisteme este mare datorita fragmentarii, iar realocarea in cazul schimbului hard-ului este facila.~~

~~Virtualizarea bazelor de date mareste performanta, usureaza administrarea si asigura flexibilitatea sistemului de calcul.~~

2. Virtualizarea retelelor.

Network virtualization is also a vital component for application development and testing environments. In the pre-production stages of the software development life cycle, network virtualization is the

process of recreating network conditions from the production - or real world - environment within the test environment.

Procesul implica imbinarea resurselor soft si hard intr-o singura retea virtuala de tip software.

Exista doua tipuri de virtualizare:

- Virtualizarea externa: îmbina resurse care sunt partea externa a rețelei de calculatoare.

- Virtualizarea interna: asigura conectivitatea de retea intre containerele interne a unui singur sistem de calcul.

Componentele unei retele virtuale sunt:

- Hardware de retea
 - o Switch-uri;
 - o Parti de retea.
- Echipamente de protectie (firewall sau echipamente load balancing);
- Retele virtuale (V-LAN) → folosite pentru a separa rețeaua la nivel de switch;
- Echipamente mobile;
- Mediul de transmisie.

Moduri de conectare placa de retea din sistemul virtual la sistemul real:

Each of the eight networking adapters can be separately configured to operate in one of the following modes:

Not attached

In this mode, VirtualBox reports to the guest that a network card is present, but that there is no connection -- as if no Ethernet cable was plugged into the card. This way it is possible to "pull" the virtual Ethernet cable and disrupt the connection, which can be useful to inform a guest operating system that no network connection is available and enforce a reconfiguration.

Network Address Translation (NAT)

If all you want is to browse the Web, download files and view e-mail inside the guest, then this default mode should be sufficient for you, and you can safely skip the rest of this section. Please note that there are certain limitations when using Windows file sharing (see [Section 6.3.3, "NAT limitations"](#) for details).

NAT Network

The NAT network is a new NAT flavour introduced in VirtualBox 4.3. See [6.4](#) for details.

Bridged networking

This is for more advanced networking needs such as network simulations and running servers in a guest. When enabled, VirtualBox connects to one of your installed network cards and exchanges network packets directly, circumventing your host operating system's network stack.

Internal networking

This can be used to create a different kind of software-based network which is visible to selected virtual machines, but not to applications running on the host or to the outside world.

Host-only networking

This can be used to create a network containing the host and a set of virtual machines, without the need for the host's physical network interface. Instead, a virtual network interface (similar to a loopback interface) is created on the host, providing connectivity among virtual machines and the host.

Generic networking

Rarely used modes share the same generic network interface, by allowing the user to select a driver which can be included with VirtualBox or be distributed in an extension pack.

At the moment there are potentially two available sub-modes:

UDP Tunnel

This can be used to interconnect virtual machines running on different hosts directly, easily and transparently, over existing network infrastructure.

VDE (Virtual Distributed Ethernet) networking

This option can be used to connect to a Virtual Distributed Ethernet switch on a Linux or a FreeBSD host. At the moment this needs compiling VirtualBox from sources, as the Oracle packages do not include it.

The "**Paravirtualized network adapter (virtio-net)**" is special. If you select this, then VirtualBox does *not* virtualize common networking hardware (that is supported by common guest operating systems out of the box). Instead, VirtualBox then expects a special software interface for virtualized environments to be provided by the guest, thus avoiding the complexity of emulating networking hardware and improving network performance. Starting with version 3.1, VirtualBox provides support for the industry-standard "virtio" networking drivers, which are part of the open-source KVM project.

The "virtio" networking drivers are available for the following guest operating systems:

- Linux kernels version 2.6.25 or later can be configured to provide virtio support; some distributions also back-ported virtio to older kernels.
- For Windows 2000, XP and Vista, virtio drivers can be downloaded and installed from the KVM project web page.^[29]

Descriere problema virtualizare:

desi exista mai multe retele virtuale, la nivel acces la retea exista o singura placa fizica, care permite accesarea datelor indiferent de retea

De ce virtualizare azi?

- Virtualizarea nu este un concept nou!
- Virtualizarea este populara acum deoarece este accesibila unui grup larg de utilizatori si administratori de sisteme
- Motive generale pentru cresterea popularitatii virtualizarii:
 - Puterea si performantele hardwareului tip x86 continua sa creasca
 - Procesoarele sunt rapide si acceseaza o memorie mare
 - Procesoarele multi-core permit mono-sistemelor sa efectueze mai multe sarcini simultan
 - Creste sansa ca hardware-ul sa fie sub-utilizat.
 - Virtualizarea ofera o modalitate excelenta de a exploata hardware-ul existent si de a reduce costurile
 - Integrarea de suport hardware pentru virtualizare in ultimele generatii de procesoare Intel si AMD, placi de baza
 - Existenta unor produse variate pentru virtualizare pentru sisteme desktop si server ce ruleaza pe sisteme tip x86 hardware.
 - Numeroase dintre acestea (like Xen) sunt open-source software
- Virtualizarea continua sa-si dovedeasca importanta in business si mediul academic

| Avantajele virtualizarii

1. Utilizarea mai buna a hardware-ului existent
2. Reducere in costurile de hardware
3. Reducerea costurilor cu infrastructura IT
4. Sistem de administrare simplificat
5. Recuperea rapida din esecuri
6. Expansiunea simpla a capacitatii
7. Suport simplu pentru aplicatii si sisteme proprietar
8. Dezvoltare simplificata la nivel de sistem
9. Sistem simplificat de instalare si lansare
10. Sistem simplificat de testare a sistemelor si aplicatiilor

CURS 8

Evolutia structurilor pentru virtualizare

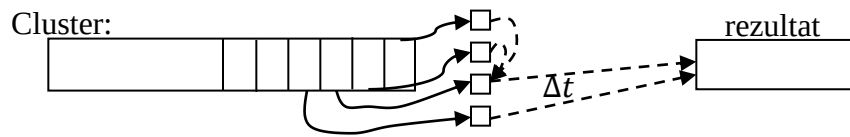
Punctele de reper sunt:

- a) Mainframe;
- b) Cluster;
- c) Grid;
- d) Cloud.

a) Mainframe

- A aparut incepand cu anii `60;
- Constau in grupuri mari de calculatoare, cu putere mare de calcul ale caror caracteristici principala era securitatea si redundanta hardware (mainframe-ul asigura o disponibilitate continua);
- Datele de intrare erau introduse prin istoric: cartele PERFORATE, apoi benzi magnetice, in prezent sistemele pot fi configurate din interfata WEB;
- Necesarul de putere s-a redus dramatic in anii `90 cand s-a realizat trecerea de la tehnologia bipolară la tehnologia CMOS;
- Scopul era executarea aplicațiilor care necesitau putere mare de calcul si aveau surse multiple de date.
- SO tipice CP/CMS

b) Cluster



Caracteristici sisteme cluster

- Sisteme strans cuplate (Tightly coupled systems);
- Creaza impresia unui singur sistem de calcul (Single system image);
- Sistem centralizat de Job management si planificare

Datorita evolutiei performantelor procesoarelor s-a dezvoltat o ramura specializata in procesarea de mare viteza folosind ca mediu de transfer intre calculatoare retelele locale. Acestea au ajuns la un nivel in care ofera latime de banda mare si latenta mica;

- Calculatoarele dintr-un cluster functioneaza impreuna ca un singur calculator;
- Exista mai multe tipuri de clustere:
 - o High availability cluster (HA) → au scopul de a imbunatati disponibilitatea serviciilor oferite;
 - o Load balancing cluster (LB) → apare cand mai multe calculatoare impart sarcinile de lucru funcționând ca un singur calculator virtual;
 - o Clustere pentru calcul (CC) → rolul lor este de a efectua calcule;
 - poate necesita comunicatii mai frecvente între noduri decat alte clustere;

În informatică FLOPS este un acronim ce provine de la expresia engleză floating point operations per second (tradus: operații în virgulă mobilă pe secundă). FLOPS reprezintă o unitate de măsură a puterii de calcul a unui calculator sau sistem de calcul, măsurând numărul maxim de operații în virgulă mobilă, (de regulă adunări și înmulțiri), ce sunt executate pe secundă. Unitatea FLOPS își găsește folosul mai ales în domeniul calculului științifice, la care calculul în virgulă mobilă este frecvent folosit. estimativ: 2010, the fastest six-core PC processor reaches 109 gigaFLOPS (Intel Core i7 980 XE; Nvidia Tesla C2050 GPU computing processors perform around 515 gigaFLOPS.

În prezent se întocmesc ierarhii cu cele mai performante calculatoare virtuale; ordinul de mărime privind performanța este de peta fops (peta= 10^{15} operații în virgula mobilă).

Exemple de cluster: Beowulf. Cateva din distribuțiile de linux care implementează acest tip de cluster: MOSIX, geared toward computationally intensive, IO-low, applications; ClusterKnoppix, based on Knoppix; Kerrighed.

c) Grid

Sistemele Grid sunt similare cu sistemele cluster pentru că fac uz de mai multe calculatoare conectate pentru a rezolva o problemă mare. O diferență este că un cluster este un sistem omogen în timp ce rețelele sunt eterogene, adică sistemele care fac parte dintr-o rețea grid pot rula sisteme de operare diferite și au hardware diferit în timp ce calculatoarele dintr-un cluster au toate același hardware și sistemul de operare.

O rețea grid poate face uz de puterea de calcul de rezervă a unui computer desktop, în timp ce mașinile dintr-un cluster sunt dedicate pentru a lucra ca o singură unitate și nimic altceva. Rețelele Grid sunt în mod inerent distribuite într-o rețea LAN, WAN sau metropolitană. Pe de altă parte, calculatoarele din cluster sunt în mod normal conținute într-un singur loc.

O altă diferență constă în modul în care resursele sunt manipulate. În cazul unui cluster, întregul sistem (toate nodurile) se comportă ca un sistem de vizualizare unică și resursele sunt gestionate de către managerul de resurse centralizate. În cazul unui Grid, fiecare nod este autonom adică are managerul de resurse proprii și se comportă ca o entitate independentă.

Avantajul principal al calculului distribuit (grid) este că fiecare nod poate fi achiziționat individual, fara restrictii de configuratie si care, atunci este combinat cu alte calculatoare in grid, poate produce o resursă de calcul similară cu a supercomputerelor, dar la un cost mai mic. Dezavantajul principal este performanta deoarece schimbul de date intre procesoarele plasate in diverse zone se poate realiza prin conexiuni care nu au conexiuni de mare viteză. Sistemele Grid sunt, așadar, potrivite pentru aplicații în care calculele multiple paralele pot avea loc independent, fără nevoia de a comunica rezultatele intermediare între procesoare. Pentru aceste aplicatii scalabilitatea rețelelor grid, dispersate geografic, este un avantaj mult mai mare decat viteza mica de conectivitate intre elementele grid-ului.

- Structura de grid identifica o conexiune între mai multe masini in scopul analizei datelor experimentale care nu pot fi analizate eficient într-o singura locatie;
- Un grid este o forma de calcul distribuit in care mai multe calculatoare sunt conectate la distanta prin retele locale sau prin internet; Diferenta fata de cluster este distribuirea pe zone mari geografice si structura eterogena a unitatilor de calcul;
- Datorita lipsei de control centralizat asupra hardware-ului modelul de procesare difera fata de cluster prin asignarea de sarcini de lucru foarte mari si prelucrarea rezultatelor dupa un anumit timp.

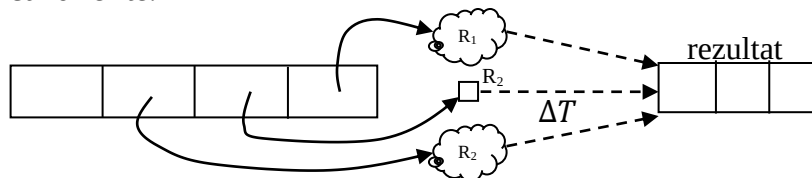
Caracteristici sisteme grid

- Loosely coupled (Slab cuplate - Decentralization)
- Diversitate si dinamism (datorata faptului ca nu exista un control unic asupra tuturor sistemelor de calcul din grid)
- Job Management si planificare distribuita

Areas of Grid Computing and it's applications for modeling and computing

1. Predictive Modeling and Simulations
2. Engineering Design and Automation
3. Energy Resources Exploration
4. Medical, Military and Basic Research
5. Visualization

Grid. Imaginea ilustreaza faptul ca sarcinile sunt distribuite între calculatoare eterogene cu puteri de calcul diferite.



$$\Delta T(grid) \gg \Delta t(cluster)$$

OBS. Acest model de calcul numit peer-to-peer cu mouting este similar cu gridul insa nodurile componente sunt utilizatori voluntari.

Modulul foloseste arhitectura client-server si se bazeaza pe numarul mare al participantilor. Nivelul de performanta atins este de ordinul Peta flops. Bitcoin network – 168.26 PFLOPS in iulie 2012 si Folding@Home – 5 PFLOPS in 2009, SETI@Home in 2010 - 730 TFLOPS .

Avantajul principal al calculului în grid este costul scăzut, deoarece pot fi folosite diferite tipuri de echipamente.

Un termen asociat este **CPU scavenging** (furtul de cicluri CPU).

De multe ori nu utilizează la întreaga capacitate echipamentele disponibile (exp. noaptea). În acest moment procesoarele pot fi „împrumutate” de grid pentru a realiza calcule până dimineața următoare. Împrumutarea se realizează fie printr-un modul integrat în SO (Windows HPC) sau aplicații dedicate (CHAOS).

Tot în anii '60 a apărut termenul de **utility computing** în care utilizatorii pot accesa printr-o rețea de disponibilitate mare resurse situate la distanță (rețeaua electrică).

Conceptul de utility computing a fost extins la rețelele de internet dând naștere la cloud.

d) Cloud

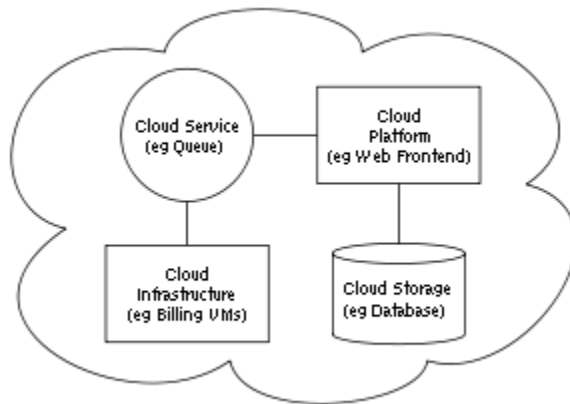
Această metodă de interconectare se bazează pe conceptul resursă software as service (SaaS). Conceptul permite închirierea serviciilor pentru o durată de timp determinată fără ca utilizatorul să necesite instalarea și configurarea de resurse hardware dedicate, în scopul

-facturării mai eficiente.

-amortizarea costurilor de evoluție hardware prin închirierea de hardware (putere de calcul);

- Un alt avantaj este că serviciile pot fi oferite la costuri reduse sau gratuit, însă împreună cu publicitate.

Arhitectura sistemelor Cloud:



Cloud computing exhibits the following key characteristics:

- **Agility** improves with users' ability to re-provision technological infrastructure resources.
- **Application programming interface** (API) accessibility to software that enables machines to interact with cloud software in the same way that a traditional user interface (e.g., a computer desktop) facilitates interaction between humans and computers. Cloud computing systems typically use Representational State Transfer (**REST**)-based APIs.
- **Cost**: cloud providers claim that computing costs reduce. A public-cloud delivery model converts capital expenditure to operational expenditure.^[31] This purportedly lowers barriers to

[entry](#), as infrastructure is typically provided by a third-party and does not need to be purchased for one-time or infrequent intensive computing tasks. Pricing on a utility computing basis is fine-grained, with usage-based options and fewer IT skills are required for implementation (in-house).^[32] The e-FISCAL project's state-of-the-art repository^[33] contains several articles looking into cost aspects in more detail, most of them concluding that costs savings depend on the type of activities supported and the type of infrastructure available in-house.

- **[Device and location independence](#)**^[34] enable users to access systems using a web browser regardless of their location or what device they use (e.g., PC, mobile phone). As infrastructure is off-site (typically provided by a third-party) and accessed via the Internet, users can connect from anywhere.^[32]
- **[Virtualization](#)** technology allows sharing of servers and storage devices and increased utilization. Applications can be easily migrated from one physical server to another.
- **[Multitenancy](#)** enables sharing of resources and costs across a large pool of users thus allowing for:
 - **centralization** of infrastructure in locations with lower costs (such as real estate, electricity, etc.)
 - **peak-load capacity** increases (users need not engineer for highest possible load-levels)
 - **utilisation and efficiency** improvements for systems that are often only 10–20% utilised.^{[11][35]}
- **[Reliability](#)** improves with the use of multiple redundant sites, which makes well-designed cloud computing suitable for [business continuity](#) and [disaster recovery](#).^[36]
- **Scalability and [elasticity](#)** via dynamic ("on-demand") [provisioning](#) of resources on a fine-grained, self-service basis near real-time,^{[37][38]} without users having to engineer for peak loads.^{[39][40][41]}
- **[Performance](#)** is monitored, and consistent and loosely coupled architectures are constructed using [web services](#) as the system interface.^{[32][42][43]}
- **[Security](#)** can improve due to centralization of data, increased security-focused resources, etc., but concerns can persist about loss of control over certain sensitive data, and the lack of security for stored kernels.^[44] Security is often as good as or better than other traditional systems, in part because providers are able to devote resources to solving security issues that many customers cannot afford to tackle.^[45] However, the complexity of security is greatly increased when data is distributed over a wider area or over a greater number of devices, as well as in multi-tenant systems shared by unrelated users. In addition, user access to security [audit logs](#) may be difficult or impossible. Private cloud installations are in part motivated by users' desire to retain control over the infrastructure and avoid losing control of information security.
- **[Maintenance](#)** of cloud computing applications is easier, because they do not need to be installed on each user's computer and can be accessed from different places.

The [National Institute of Standards and Technology](#)'s definition of cloud computing identifies "five essential characteristics":

On-demand self-service. A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service provider.

Broad network access. Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, tablets, laptops, and workstations).

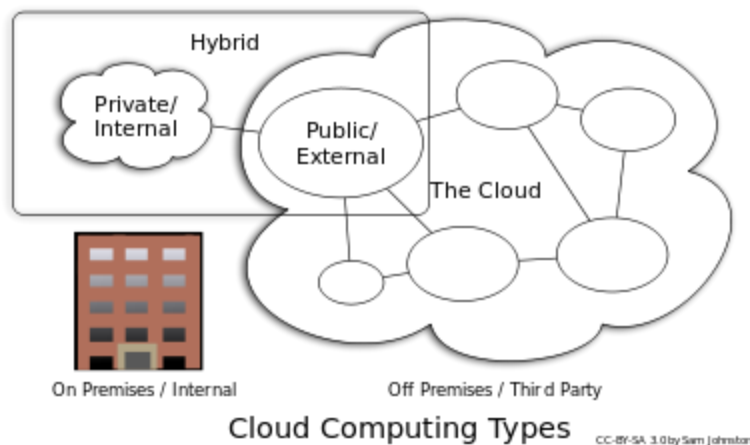
Resource pooling. The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand.

Rapid elasticity. Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear unlimited and can be appropriated in any quantity at any time.

Measured service. Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.

—National Institute of Standards and Technology^[2]

Tipuri de sisteme Cloud:



Private cloud

Private cloud is cloud infrastructure operated solely for a single organization, whether managed internally or by a third-party, and hosted either internally or externally.^[2] Undertaking a private cloud project requires a significant level and degree of engagement to virtualize the business environment, and requires the organization to reevaluate decisions about existing resources. When done right, it can improve business, but every step in the project raises security issues that must be addressed to prevent serious vulnerabilities.^[60] Self-run data centers^[61] are generally capital intensive. They have a significant physical footprint, requiring allocations of space, hardware, and environmental controls. These assets have to be refreshed periodically, resulting in additional capital expenditures. They have attracted criticism because users "still have to buy, build, and manage them" and thus do not benefit from less hands-on management,^[62] essentially "[lacking] the economic model that makes cloud computing such an intriguing concept".^{[63][64]}

Public cloud

A cloud is called a "public cloud" when the services are rendered over a network that is open for public use. Public cloud services may be free or offered on a pay-per-usage model.^[65] Technically there may be little or no difference between public and private cloud architecture, however, security consideration may be substantially different for services (applications, storage, and other resources) that are made available by a service provider for a public audience and when communication is effected over a non-trusted network. Generally, public cloud service providers like Amazon AWS, Microsoft and Google own and operate the infrastructure at their [data center](#) and access is generally via the Internet. AWS and Microsoft also offer direct connect services called "AWS Direct Connect" and "Azure ExpressRoute" respectively, such connections require customers to purchase or lease a private connection to a peering point offered by the cloud provider.^[34]

Hybrid cloud

Hybrid cloud is a composition of two or more clouds (private, community or public) that remain distinct entities but are bound together, offering the benefits of multiple deployment models. Hybrid cloud can also mean the ability to connect collocation, managed and/or dedicated services with cloud resources.^[2]

[Gartner, Inc.](#) defines a hybrid cloud service as a cloud computing service that is composed of some combination of private, public and community cloud services, from different service providers.^[66] A hybrid cloud service crosses isolation and provider boundaries so that it can't be simply put in one category of private, public, or community cloud service. It allows one to extend either the capacity or the capability of a cloud service, by aggregation, integration or customization with another cloud service.

Varied use cases for hybrid cloud composition exist. For example, an organization may store sensitive client data in house on a private cloud application, but interconnect that application to a business intelligence application provided on a public cloud as a software service.^[67] This example of hybrid cloud extends the capabilities of the enterprise to deliver a specific business service through the addition of externally available public cloud services.

Another example of hybrid cloud is one where IT organizations use public cloud computing resources to meet temporary capacity needs that can not be met by the private cloud.^[68] This capability enables hybrid clouds to employ cloud bursting for scaling across clouds.^[2] Cloud bursting is an application deployment model in which an application runs in a private cloud or data center and "bursts" to a public cloud when the demand for computing capacity increases. A primary advantage of cloud bursting and a hybrid cloud model is that an organization only pays for extra compute resources when they are needed.^[69] Cloud bursting enables data

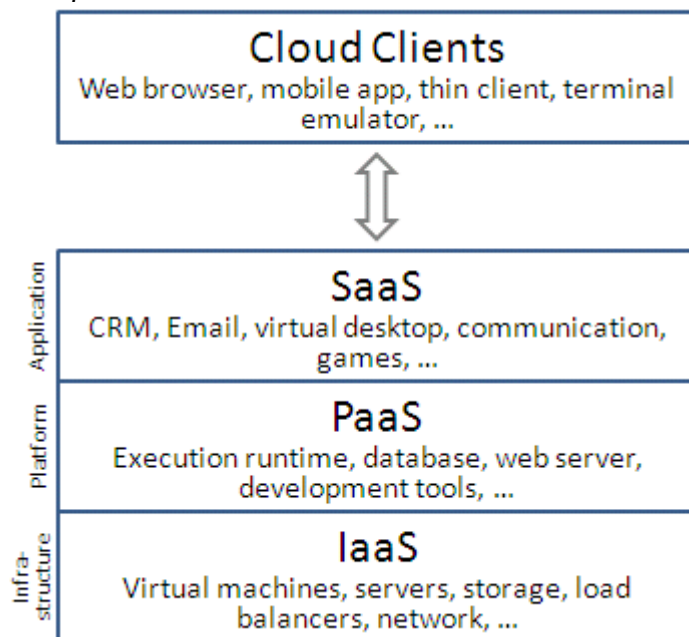
Fog computing/ Edge Computing

In 2011, the need to extend cloud computing with fog computing emerged, in order to cope with huge number of IoT devices and big data volumes for real-time low-latency applications.^[5] Fog computing, also called edge computing, is intended for distributed computing where numerous "peripheral" devices connect to a [cloud](#). The word "fog" refers to its cloud-like properties, but closer to the "ground", i.e. IoT devices.^[6] Many of these devices will generate voluminous raw data (e.g., from sensors), and rather than forward all this data to cloud-based servers to be processed, the idea behind fog computing is to do as much processing as possible using computing units co-located with the data-generating devices, so that processed rather than raw data is forwarded, and bandwidth requirements are

reduced. An additional benefit is that the processed data is most likely to be needed by the same devices that generated the data, so that by processing locally rather than remotely, the latency between input and response is minimized. This idea is not entirely new: in non-cloud-computing scenarios, special-purpose hardware (e.g., signal-processing chips performing [Fast Fourier Transforms](#)) has long been used to reduce latency and reduce the burden on a CPU.

Fog networking consists of a [control plane](#) and a [data plane](#). For example, on the data plane, fog computing enables computing services to reside at the edge of the network as opposed to servers in a data-center. Compared to [cloud computing](#), fog computing emphasizes proximity to end-users and client objectives (e.g. operational costs, security policies,^[7] resource exploitation), dense geographical distribution and context-awareness (for what concerns computational and IoT resources), latency reduction and backbone bandwidth savings to achieve better [quality of service](#) (QoS)^[8] and edge analytics/stream mining, resulting in superior user-experience^[9] and redundancy in case of failure while it is also able to be used in [Assisted Living](#) scenarios.

Clasificare servicii de cloud:



CURS 10

Sisteme Cloud

History has a funny way of repeating itself, or so they say. But it may come as some surprise to find this old cliché applies just as much to [the history of computers](#) as to wars, revolutions, and kings and queens. For the last three decades, one trend in computing has been loud and clear: big, centralized, mainframe systems have been "out"; personalized, power-to-the-people, do-it-yourself PCs have been "in." Before personal computers took

off in the early 1980s, if your company needed sales or payroll figures calculating in a hurry, you'd most likely have bought in "data-processing" services from another company, with its own expensive computer systems, that specialized in number crunching; these days, you can do the job just as easily on your desktop with off-the-shelf software. Or can you? In a striking throwback to the 1970s, many companies are finding, once again, that buying in computer services makes more business sense than do-it-yourself. This new trend is called cloud computing and, not surprisingly, it's linked to the [Internet](#)'s inexorable rise. What is cloud computing? How does it work? Let's take a closer look!

Photo: Cloud computing: the hardware, software, and applications you're using may be anywhere up in the "cloud." As long as it all does what you want, you don't need to worry where it is or how it works. Composite photo by Explainthatstuff.com based on a picture of an IBM Blue Gene/P [supercomputer](#), by courtesy of [Argonne National Laboratory](#), published under a [Creative Commons Licence](#), and clouds photographed somewhere over Dorset, England, in 2011.

What is cloud computing?

Cloud computing means that instead of all the [computer](#) hardware and software you're using sitting on your desktop, or somewhere inside your company's [network](#), it's provided for you as a service by another company and accessed over the [Internet](#), usually in a completely seamless way. Exactly where the hardware and software is located and how it all works doesn't matter to you, the user—it's just somewhere up in the nebulous "cloud" that the Internet represents.

Cloud computing is a buzzword that means different things to different people. For some, it's just another way of describing IT (information technology) "outsourcing"; others use it to mean any computing service provided over the Internet or a similar network; and some define it as any bought-in computer service you use that sits outside your firewall. However we define cloud computing, there's no doubt it makes most sense when we stop talking about abstract definitions and look at some simple, real examples—so let's do just that.

Simple examples of cloud computing

Most of us use cloud computing all day long without realizing it. When you sit at your PC and type a query into Google, the computer on your desk isn't playing much part in finding the answers you need: it's no more than a messenger. The words you type are swiftly shuttled over the Net to one of Google's hundreds of thousands of [clustered PCs](#), which dig out your results and send them promptly back to you. When you do a Google search, the real work in finding your answers might be done by a computer sitting in California, Dublin, Tokyo, or Beijing; you don't know—and most likely you don't care!

The same applies to Web-based email. Once upon a time, email was something you could only send and receive using a program running on your PC (sometimes called a mail

client). But then Web-based services such as Hotmail came along and carried email off into the cloud. Now we're all used to the idea that emails can be stored and processed through a server in some remote part of the world, easily accessible from a Web browser, wherever we happen to be. Pushing email off into the cloud makes it supremely convenient for busy people, constantly on the move.

Preparing documents over the Net is a newer example of cloud computing. Simply log on to a web-based service such as Google Documents and you can create a document, spreadsheet, presentation, or whatever you like using Web-based software. Instead of typing your words into a program like Microsoft Word or OpenOffice, running on your computer, you're using similar software running on a PC at one of Google's world-wide data centers. Like an email drafted on Hotmail, the document you produce is stored remotely, on a Web server, so you can access it from any Internet-connected computer, anywhere in the world, any time you like. Do you know where it's stored? No! Do you care where it's stored? Again, no! Using a Web-based service like this means you're "contracting out" or "outsourcing" some of your computing needs to a company such as Google: they pay the cost of developing the software and keeping it up-to-date and they earn back the money to do this through advertising and other paid-for services.

What makes cloud computing different?

It's managed

Most importantly, the service you use is provided by someone else and managed on your behalf. If you're using Google Documents, you don't have to worry about buying umpteen licenses for word-processing software or keeping them up-to-date. Nor do you have to worry about viruses that might affect your computer or about backing up the files you create. Google does all that for you. One basic principle of cloud computing is that you no longer need to worry how the service you're buying is provided: with Web-based services, you simply concentrate on whatever your job is and leave the problem of providing dependable computing to someone else.

It's "on-demand"

Cloud services are available on-demand and often bought on a "pay-as-you go" or subscription basis. So you typically buy cloud computing the same way you'd buy electricity, telephone services, or Internet access from a utility company. Sometimes cloud computing is free or paid-for in other ways (Hotmail is subsidized by advertising, for example). Just like electricity, you can buy as much or as little of a cloud computing service as you need from one day to the next. That's great if your needs vary unpredictably: it means you don't have to buy your own gigantic computer system and risk have it sitting there doing nothing.

It's public or private

Now we all have PCs on our desks, we're used to having complete control over our computer systems—and complete responsibility for them as well. Cloud computing changes all that. It comes in two basic flavors, public and private, which are the cloud equivalents of the Internet and Intranets. Web-based email and free services like the ones Google provides are the most familiar examples of public clouds. The world's biggest online retailer, Amazon, became the world's largest provider of public cloud computing in early 2006. When it found it was using only a fraction of its huge, global, computing power, it started renting out its spare capacity over the Net through a new entity called Amazon Web Services. Private cloud computing works in much the same way but you access the resources you use through secure network connections, much like an Intranet. Companies such as Amazon also let you use their publicly accessible cloud to make your own secure private cloud, known as a Virtual Private Cloud (VPC), using virtual private network (VPN) connections.

Types of cloud computing

IT people talk about three different kinds of cloud computing, where different services are being provided for you. Note that there's a certain amount of vagueness about how these things are defined and some overlap between them.

- Infrastructure as a Service (IaaS) means you're buying access to raw computing hardware over the Net, such as servers or storage. Since you buy what you need and pay-as-you-go, this is often referred to as utility computing. Ordinary web hosting is a simple example of IaaS: you pay a monthly subscription or a per-megabyte/gigabyte fee to have a hosting company serve up files for your website from their servers.
- Software as a Service (SaaS) means you use a complete application running on someone else's system. Web-based email and Google Documents are perhaps the best-known examples. Zoho is another well-known SaaS provider offering a variety of office applications online.
- Platform as a Service (PaaS) means you develop applications using Web-based tools so they run on systems software and hardware provided by another company. So, for example, you might develop your own ecommerce website but have the whole thing, including the shopping cart, checkout, and payment mechanism running on a merchant's server. App Cloud (from salesforce.com) and the Google App Engine are examples of PaaS.

Advantages and disadvantages of cloud computing

What's good and bad about cloud computing?

Advantages

The pros of cloud computing are obvious and compelling. If your business is selling books or repairing shoes, why get involved in the nitty gritty of buying and maintaining a complex computer system? If you run an insurance office, do you really want your sales agents wasting time running anti-virus software, upgrading word-processors, or worrying about hard-drive crashes? Do you really want them cluttering your expensive computers with their personal emails, illegally shared [MP3](#) files, and naughty YouTube videos—when you could leave that responsibility to someone else? Cloud computing allows you to buy in only the services you want, when you want them, cutting the upfront capital costs of computers and peripherals. You avoid equipment going out of date and other familiar IT problems like ensuring system security and reliability. You can add extra services (or take them away) at a moment's notice as your business needs change. It's really quick and easy to add new applications or services to your business without waiting weeks or months for the new computer (and its software) to arrive.

Drawbacks



Instant convenience comes at a price. Instead of purchasing computers and software, cloud computing means you buy services, so one-off, upfront capital costs become ongoing operating costs instead. That might work out much more expensive in the long-term.

If you're using software as a service (for example, writing a report using an online word processor or sending emails through webmail), you need a reliable, high-speed, [broadband](#) Internet connection functioning the whole time you're working. That's something we take for granted in countries such as the United States, but it's much more of an issue in developing countries or rural areas where broadband is unavailable.

If you're buying in services, you can buy only what people are providing, so you may be restricted to off-the-peg solutions rather than ones that precisely meet your needs. Not only that, but you're completely at the mercy of your suppliers if they suddenly decide to stop supporting a product you've come to depend on. (Google, for example, upset many

users when it [announced](#) in September 2012 that its cloud-based Google Docs would drop support for old but *de facto* standard Microsoft Office file formats such as .DOC, .XLS, and .PPT, giving a mere *one week's notice* of the change—although, after public pressure, it later extended the deadline by three months.) Critics charge that cloud-computing is a return to the bad-old days of mainframes and proprietary systems, where businesses are locked into unsuitable, long-term arrangements with big, inflexible companies. Instead of using "generative" systems (ones that can be added to and extended in exciting ways the developers never envisaged), you're effectively using "dumb terminals" whose uses are severely limited by the supplier. Good for convenience and security, perhaps, but what will you lose in flexibility? And is such a restrained approach good for the future of the Internet as a whole? (To see why it may not be, take a look at Jonathan Zittrain's eloquent book [The Future of the Internet—And How to Stop It.](#))

Think of cloud computing as renting a fully serviced flat instead of buying a home of your own. Clearly there are advantages in terms of convenience, but there are huge restrictions on how you can live and what you can alter. Will it automatically work out better and cheaper for you in the long term?

Sisteme Cluster SSI

Single system image (SSI)

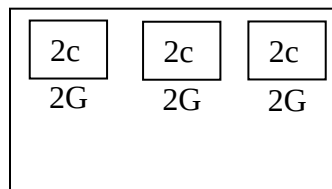
Un cluster este un grup de calculatoare functionand unitar si care se comporta in multe aspecte ca un singur calculator. In majoritatea cazurilor clustererele sunt conectate la retele de calculatoare.

Cele trei tipuri de clusterere sunt:

- High availability cluster (HA) – stau online mereu;
- Load balancing cluster (LB) – rol de a balansa incarcarea echipamentelor;
- HPC Cluster pentru calcul (pentru procesare).

Clustererele de tipul SSI ruleaza un SO unitar care repartizeaza sarcinile prin intermediul functiilor specifice pe diferite noduri.

SSI cluster:
6 core-uri
6G memorie



Comportamentul este apropiat de cel al SO distribuite, insa functiile oferite sunt mai restranse fata de un SO distribuit.

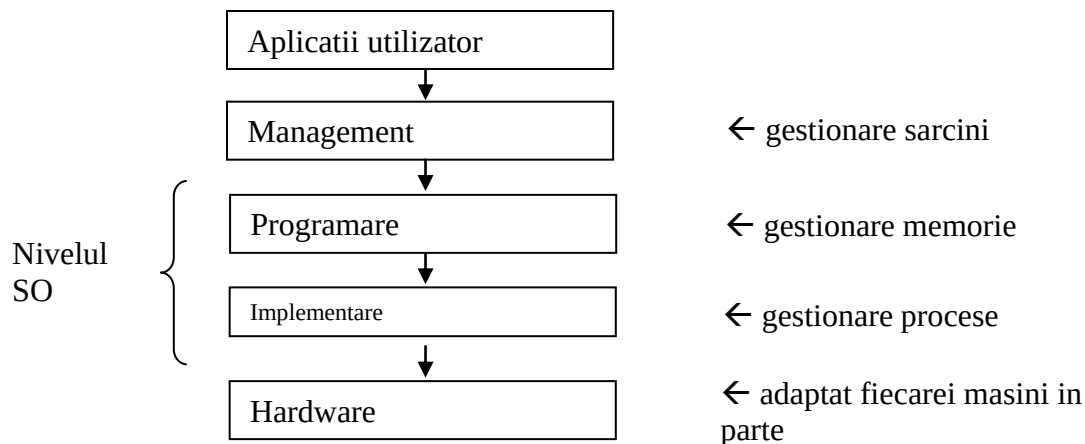
Principala activitate gestionata este migrarea proceselor si gestorarea resurselor.

Avantajele clusterelor SSI:

- Punct unic de intrare pentru utilizatorii sau procesele care se conecteaza la cluster;
- Interfata utilizator unica (oferind inclusiv interfata grafica);

- Spatiu unic de adrese pentru procese (implica identificatori unici la nivelul clusterului pentru procese simplificand comunicatia intre acestia);
- Spatiu unic de intrare-iesire;
- Sistem unic de fisiere (deobicei NFS);
- Gestionarea unitara;
- Salvarea unitara a starilor intermediare.

Se disting urmatoarele niveluri in arhitectura unui cluster:



1. Nivelul hardware preia resursele fiecarui calculator si le pune la dispozitia SO al clusterului.
 - Transmisia intre noduri poate fi broadcast sau unicast;
 - Se asigura mecanisme pentru detectia erorilor in situatia in care anumite statii devin indisponibile.
2. Nivelul SO
 - Scopul este paralelizarea aplicatiilor secventiale;
 - Rolul este planificarea proceselor, identificarea resurselor libere, migrarea proceselor si balansarea incarcarii (low balancing). Accesarea acestor functii se face fara comenzi suplimentare.
3. Nivelul middleware (management)
 - Se interpune intre SO si aplicatiile utilizator cu rol de management al sarcinilor care sunt executate;
 - Planificarea globala a sarcinilor ofera un nivel ridicat de transparenta utilizatorului.
4. Nivelul aplicatie
 - Este nivelul ce ruleaza aplicatiile utilizator si interfata grafica (GUI) a SO.

Studiu de caz

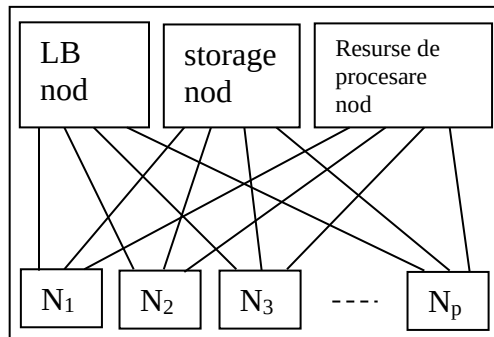
Linux Virtual Server (LVS)

- Are arhitectura transparenta pentru utilizator care ascunde existenta calculatoarelor interconectate;
- Selectarea serverelor se realizeaza prin 4 algoritmi:
 - Round Robin (RR);
 - Weighted Round Robin (WRR);
 - Least Connections (LC);
 - Weighted Least Connections (WLC).

Least connections

- Cele mai putine conexiuni;
- Algoritmul alocă sarcinile in functie de numarul de conexiuni existente.

Structura acestui cluster SSI este urmatoare:



nodStorage = stocheaza sistemul de fisiere pe cluster.

nod de procesare = implementeaza serviciile oferite de cluster.

Nodul cu resursele de procesare este cel care implementeaza serviciile oferite de cluster. Mod de functionare:

- Nodul LB primeste cererile din internet, selecteaza serverele din lista citita de la nodul cu resurse de procesare;
- Mentine starea conexiunilor concurente / paralele;
- Transmite pachete procesate in interiorul retelei.

SO mentine un serviciu de monitorizare a intregului cluster. Acesta este stocat pe **nodul cu resursele de procesare** si trimite pachete ping la anumite intervale de timp catre nodurile din reseaua locala a clusterului. Daca nodul nu raspunde, el este eliminat din lista cu resurse de procesare, dar si din lista nodului LB si a celui de gestionare storage.

Pentru a preveni functionarea incorecta a nodului de LB, de obicei acesta e dublat de un nod de back-up.

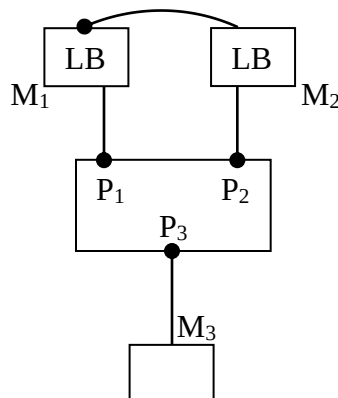
In cazul in care nodul principal executa o operatie eronata, serverul de back-up realizeaza operatia de impersonare ARP (ARP spoofing) pentru a prelua rolul / adresa IP a LB / calculatorului principal.

Cand nodul principal a redevenit functional el contacteaza nodul de back-up si acesta elibereaza adresa IP.

Operatia de preluare a adresei IP duce la terminarea conexiunilor curente, deci clientii vor retransmite cererile catre cluster.

ARP = adress resolution protocol.

Operatia de ARP spoofing:



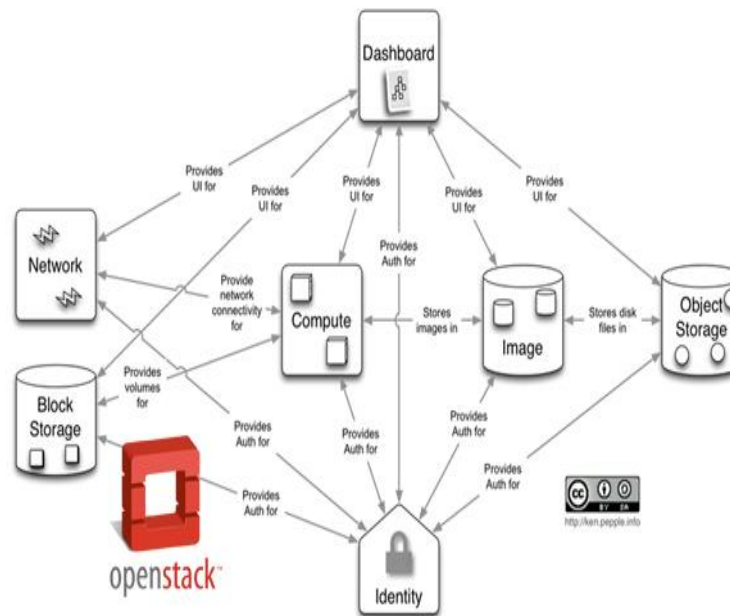
Operatia ARP spoofing se bazeaza pe proprietatea switchurilor de a lucra doar cu adrese MAC.

Inainte	Dupa
P ₁ -M ₁	
P ₂ -M ₂	P ₃ -M ₃
P ₃ -M ₃	

Cererile ulterioare ale M₃ vor fi directionate transparent catre portul P₂.

Cateva sisteme de tipul cluster SSI sunt: OPEN SSI, OPEN MOSIX si KERRIGHED.

Openstack



In the previous post we had a quick OpenStack tutorial and intro to [What is OpenStack](#). In this post, we will be outlining in short the different OpenStack components, and how they work together. From OpenStack Cinder through OpenStack Heat, and Trove – this is your quick wiki for everything OpenStack compute, networking, and storage.

Components of Openstack

There are basically eleven components of OpenStack (two of which were just included in the last Icehouse release), below is a quick breakdown of what they are called in OpenStack speak, and what they do.

*Cloudify 3.0 is coming...taking OpenStack
Native to a whole new level. Test drive
Cloudify.* [Go](#)

Component Name	Description
OpenStack Compute (Nova)	OpenStack compute (codename: Nova) is the component that allows the user to create and manage virtual servers using machine images. It is the brain of the Cloud. OpenStack provisions and manages large networks of virtual machines.
Block Storage (Cinder)	This component provides persistent block storage to multiple instances. The flexible architecture makes creating and managing block storage devices very easy.
Object Storage (Swift)	This component stores and retrieves unstructured data through the HTTP based APIs. Further, it is also fault tolerant with its data replication and scale out architecture.

OpenStack Networking (Neutron)	It is a pluggable, scalable and API-driven system for managing networks. OpenStack networking is useful for VLAN management of IP addresses to different VMs and managing firewalls using these components.
Identity Service (Keystone)	This provides a central directory of users mapped to the services. It is used to provide an authentication and authorization service for other OpenStack services.
OpenStack Image Service (Glance)	This provides the discovery, registration and delivery of the disk and server images. It stores and retrieves the machine disk image.
OpenStack Telemetry Service (Ceilometer)	It monitors the usage of the Cloud services and decides accordingly. This component is also used to decide the usage and obtain the statistics regarding the usage.
Dashboard (Horizon)	This component provides a web-based portal to interact with the underlying OpenStack services, such as NOVA, Neutron, etc.
Orchestration Heat	This component manages multiple Cloud applications using the OpenStack-native REST API and a CloudFormation-compatible Query API.
Database as a Service (Trove)	Trove is Database as a Service for OpenStack. It's designed entirely on OpenStack , with the goal of allowing users to easily utilize the features of a relational database without the burden of handling complex administrative tasks. Cloud database administrators can provision and manage multiple database instances as needed. Initially, the service will provide resource isolation at high performance while handling complex administrative tasks including deployment, configuration, patching, backups, restores, and monitoring.

Messaging as a Service (Marconi)	Marconi is a cloud messaging and notification service for developers building applications on top of OpenStack. It features a web-friendly HTTP API, which developers can use to send messages between the various components of their OpenStack mobile applications, using a variety of communication protocols. Underlying the API is an efficient messaging engine designed with scalability and security in mind.

OpenStack has a modular architecture with various code names for its components:

- *Nova (Compute)*
- *Swift (Object Storage)*
- *Cinder (Block Storage)*
- *Glance (Image Service)*
- *Neutron (Networking)*
- *Horizon (Dashboard)*
- *Ceilometer (Telemetry)*
- *Heat (Orchestration)*

Keystone (Identity Service)

It's the **main authentication and authorization service** as I heard from someone, it's the **who are you and what do you want service**. It authorizes

- *Users*
- *Services*
- *Endpoints*

Keystone uses **tokens** for authorization and **maintains Session state**

Nova (Compute)

Nova compute or the king service **provides a platform** on which we are going to **run our guest machines**; It's the **virtual machine provisioning and management module** that defines drivers that interact with underlying virtualization. It provides a **Control plane** for an underlying hypervisors. Each hypervisor requires a separate Nova Instance. Nova **supports almost all hypervisors** known to man.

Glance (Image Service)

In simple words glance is the **Image Registry**, it stores and Manage our guest (VM) images, Disk Images, snap shots etc. It also **contains prebuilt VM templates** so that you can try it on the fly. Instances are booted from our glance image registry. **User can create custom images and upload them** to Glance later reuse. A feature of Glance is **to store images remotely so to save local disk space**.

Swift (Object Storage)

Swift Offers **cloud storage software**, Look at it as **Dropbox or Google drive**, as they are **not attached to servers**, they are individual addressable objects. It's built for scale and optimized for durability, availability, and concurrency across the entire data set. Swift is ideal for storing unstructured data that can grow without bound. Swift provides redundancy checksum for files, Files are stored as segments and a manifest file tracks them.

Cinder (Block Storage)

Cinder is also one of the **storage modules** of Openstack; **Think of it as an external hard drive or like a USB device**. It has the performance characteristics of a Hard drive but much slower than Swift and has low latency. Block Volumes are created in Swift and **attached to running Volumes** for which you want to attach an extra partition or for copying data to it. It survives the termination of an Instance. It is used to keep persistence storage. Cinder Images are mostly stored on our shared storage environment for ready availability. These Images can be clones and snapshots which can be turned into bootable images. It's similar to Amazon Elastic Block Storage.

Neutron (Networking)

Neutron, the **networking component** which was formally called Quantum. This component provides the **software defined Networking Stack** for Openstack. It provides **networking as a service**. It gives the cloud tenants an API to build rich networking topologies, and configure advanced network policies in the cloud. It enables innovation plugins (open and closed source) that introduce advanced network capabilities which let anyone **build advanced network services** (open and closed source) that plug into Openstack tenant networks means that you can **create advanced managed switches and routers**. You can even **create an intelligent switch from a PC** (Yes you can use it as a standalone component) and use it **to replace your managed switch** or at least make it act as a backup Switch.

The Horizon (explained later) has a GUI support for

- Neutron L2 and L3 network and subnet creation/deletion
- Booting VMs on specific Neutron networks.

Heat (Orchestration)

It creates a human and machine-accessible service for **managing the entire lifecycle of infrastructure and applications** within Openstack clouds. It **contains human readable templates** with simple instructions that are read by the Heat Engine. Heat along with ceilometer (explained below) can create an auto-scaling cloud.

Telemetry (Ceilometer)

This module is **responsible for metering information**. It can be used **generate bills** and based on the statistics of usage. Its API can be used with external billing systems. Administrators can create certain alarms that are triggered based on performance statistics.

Horizon (Dashboard)

Horizon is the **Dashboard to Openstack, your eyes and ears**. It provides a web-based user interface to OpenStack services including Nova, Swift, Keystone etc.

Process Workflow

Below is an example of a user who wants to host SUSE Linux. The workflow diagrams given below describe how the process flows in OpenStack.

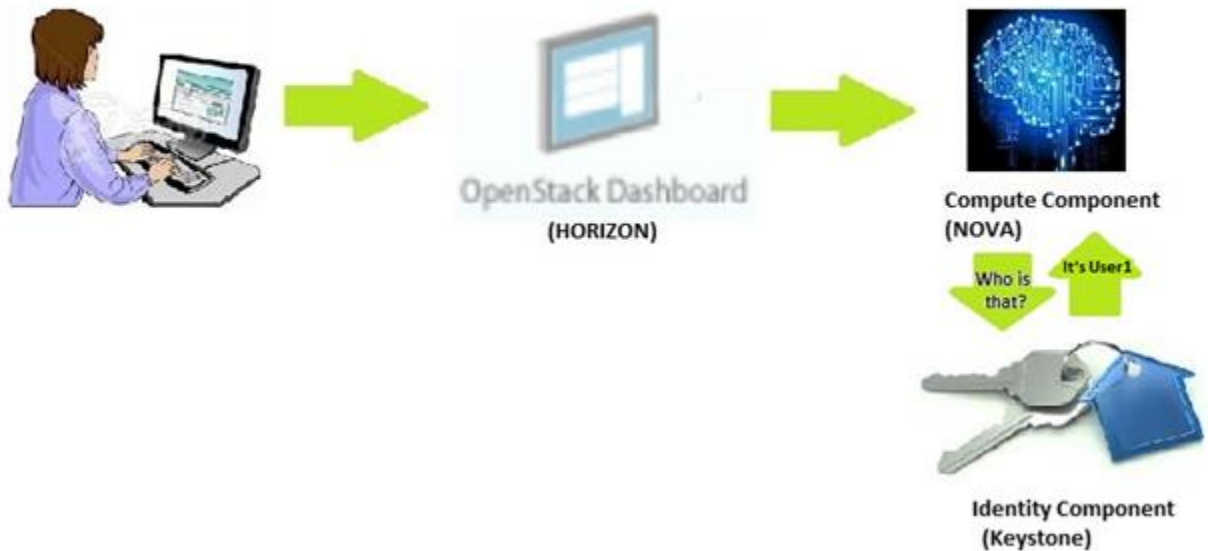
User Requests a VM running SUSE Linux



The Dashboard (horizon) passes the request to the Compute Component (NOVA)



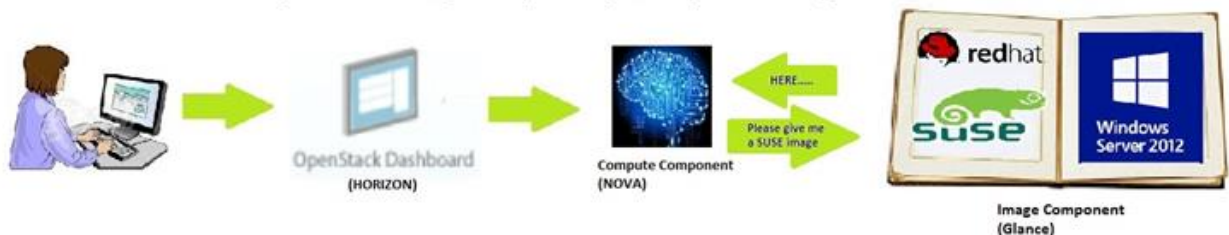
The request goes to Identity Component (keystone) for authentication



NOVA requests the Networking Component (Neutron) for an IP Address



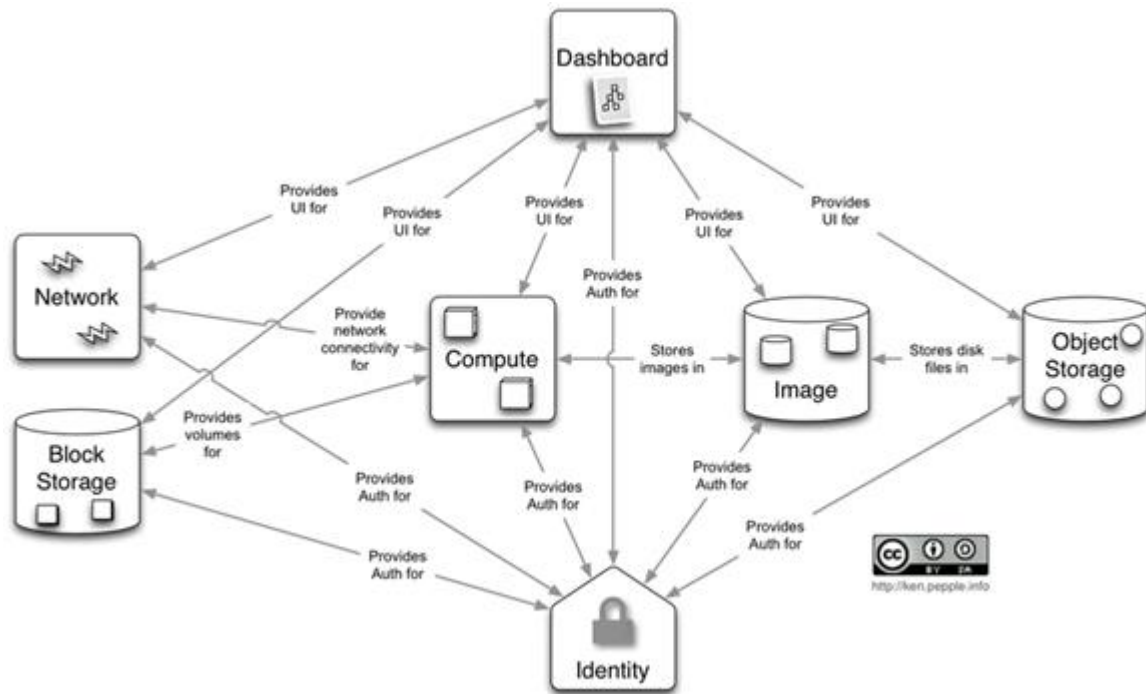
Nova requests the image component (Glance) for an image of SUSE linux



Finally after getting the image, Nova mounts it on a VM host. During the boot process of the VM, it requests Neutron (DHCP component) for an IP address.

Summary Diagram

The diagram given below depicts the intercommunication between all the components of OpenStack.



We'll be bring you more OpenStack 101 posts – so stay tuned.

CURS 11

Retele CAN (Controller Area Network)

Aceste retele au fost create la jumatatea anilor '80, in industria automobilelor din Germania pentru a permite realizarea de comutatii seriale rezistente la erori.

Protocolul folosit de retelele CAN s-a raspandit rapid nu doar in industria automobilelor ci si in alte domenii, de exemplu industria echipamentelor medicale si a aparatelor de testare.

Aceste protocol foloseste transmisii pe doua fire. Viteza de transmisie a fost marita de la prima versiune 1.0 pana la cea de-a doua versiune 2.0 de 8 ori atingand viteza de 1Mb/sec.

In 1986 cand a aparut primul automobil cu retea CAN, lungimea firelor necesare cablarii masinilor a fost redusa cu 2 km rezultand intr-un castig de 50 kg.

In 2008, peste 90% din masini utilizeaza in interior protocolul CAN

Reteaua CAN functioneaza in straturile de jos ale nivelului OSI: legatura de date si fizic. Din acest motiv, o mare parte a protocolului e dependeta de mediul de transmisie insa exista si o portiune independenta de mediul de transmisie.

...
LLC
MAC
Fizic

Protocoalele de nivel superior vor fi implementate de producatorul software.

Protocolul CAN a fost optimizat sa transmita cantitati mici de date, spre deosebire de Ethernet sau USB care sunt proiectate pentru a transmite blocuri mari de date.

Protocolul este orientat pe mesaje si nu pe adrese, realizand astfel comunicatii unicast si multicast fara a modifica tipul mesajului.

Scopul protocolului CAN este realizarea comunicatiei rapide si sigure.

Comunicare prin mesaje.

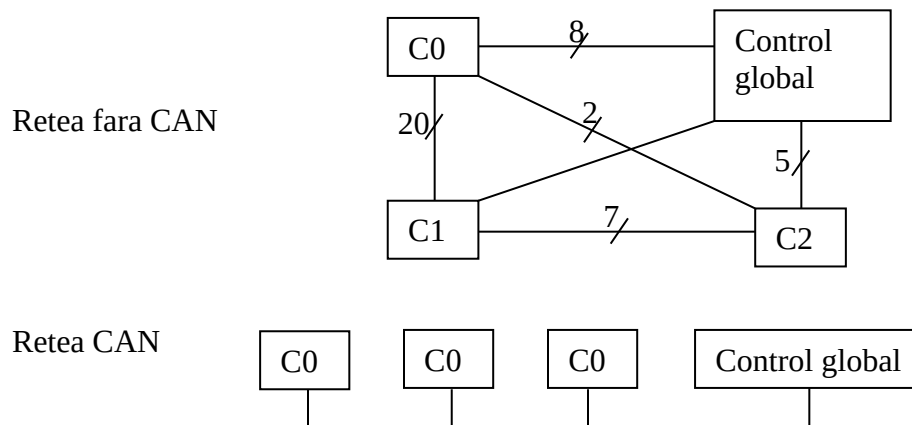
Protocolul CAN foloseste mesaje si nu foloseste adrese (Ethernetul este un protocol ce se bazeaza pe adrese) → toate nodurile din sistem primesc mesajul si fiecare nod decide daca mesajul va fi procesat sau eliminat. Un mesaj poate fi destinat unuia sau mai multor noduri in functie de proiectarea retelei.

Exp. Senzorul de airbag poate transmite un mesaj care va fi receptionat de mai multe controllere.

Protocolul CAN implementeaza o functie numita Remote Transmit Request prin care un nod poate solicita date de raspuns de la un alt nod.

Caracteristicile protocolului CAN

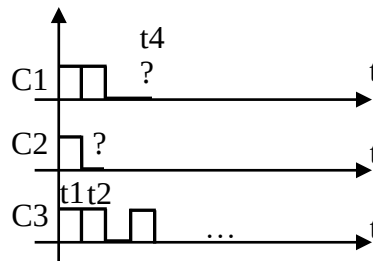
Algoritmul de tratare a coliziunilor foloseste CSMA/CD pentru a depista coliziunile. Acest protocol precizeaza ca fiecare nod din retea trebuie sa monitorizeze mediul de transmisie pentru a depista o perioada de inactivitate inainte de realizarea transmisiei. Daca o perioada de inactivitate este determinata, oricare nod are prioritate egala de transmisie. Daca doua noduri transmit simultan, este detectata coliziunea si se iau masuri corespunzatoare.



Spre deosebire de algoritmi folositi in retelele ethernet, protocolul CAN **implementeaza o metoda nedestructiva privind mesajele transmise la aparitia coliziunii, adica mesajele raman intacte chiar daca coliziunea a fost detectata.**

Protocolul CAN foloseste doua tipuri de logica: master si slave, in care semnalul / bitul de 1 este master el avand efect prioritar asupra bitului de 0 care este slave. Daca trei controllere actioneaza simultan mediul de transmisie, lucrurile se intampla in modul urmator: fiecare controller are asociat un identificator. Acesta defineste prioritatea de transmisie a dispozitivului.

controller	Cod
C1	1100
C2	1000
C3	1101



La momentul t2, controllerul C2 depisteaza faptul ca semnalul din linie nu corespunde cu semnalul transmis. Aceasta eroare nu este semnalizata deoarece presupune ca un nod prioritar transmite simultan.

La momentul de timp t4, controllerul c1 realizeaza ca un controller prioritar transmite simultan si isi inceteaza transmisia.

C3 fi singurul dintre cele trei care va termina de transmis datele deoarece are proritate maxima.

Structura frame-ului CAN

Unitatea de protocol se numeste cadru / frame.

	11 biti	18 biti			max 8 octeti			
S	ID	ID2	L	RTR	DATE	CRC 15 biti	A	E
O							C	O
F							K	F

SOF=start of frame

EOF=end of frame

ID=identificator pe 11 biti determina prioritatea mesajului. In unele situatii se folosesc 18 biti aditionali pentru identificarea prioritatii in functie de proiectarea sistemului.

L=dimensiunea campului de date

RTR=remote transmit request

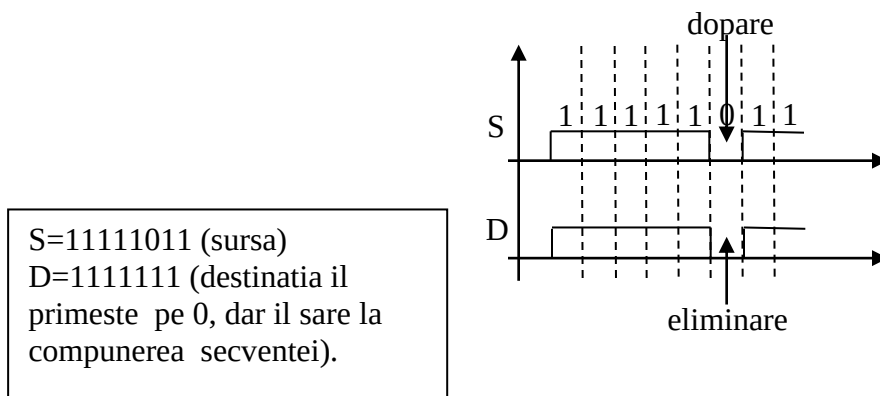
Ack este folosit pentru confirmarea receptiei; in cazul in care nu se receptioneaza de la destinatie un acknowledge, mesajul este retransmis. In multe sisteme, mecanismul de Ack este dezactivat pentru a reduce numarul de transmisii din sistem.

Detectia erorilor folosind protocolul CAN

1. Detectie prin CRC.
 - Daca se receptioneaza un frame cu CRC incorect e generata o eroare corespunzatoare;
 - Daca cel putin un nod nu a primit mesajul corect, acesta este retransmis.
2. Eroare de confirmare.
 - Daca bitul de Ack contine valoarea 1, mesajul a fost receptionat corect; valoarea 0 indica receptia incorecta.
3. Eroarea de format.
 - Daca oricare nod detecteaza un bit 1 in oricare din campurile de delimitare a frame-urilor (SOF, EOF), o eroare de format este transmisa si mesajul original este retransmis.
4. Eroarea de bit.
 - Apare in cazul in care valoarea transmisa este diferita de valoarea monitorizata in linia de date.
5. Eroare de dopare.

Deoarece CAN foloseste metoda NRZ (pe durata intregului bit semnalul nu-si modifica starea) pentru transmisie, este posibila aparitia unor secvente lungi de 0 sau 1.

Doparea cu biti inseamna adaugarea unui bit suplimentar dupa un numar bine stabilit de biti cu aceeasi valoare.



Exista si alte erori detectate de protocolul CAN, majoritatea fiind legate de starile in care se afla magistrala de comunicatie.